Networking Technologies and Applications

Cours et méthodes de calcul, séances 1 à 6

BMEVITMAC09, automne 2026. Rédigé le 2026-09-28 (séances 1 à 3), complété le 2026-10-17 (séances 4 à 6).

À retenir

Midterm (ZH)	jeudi 2026-10-29, 8h15, IE218, à la place de la séance pratique ; réussi ou échoué
Format	6 questions ouvertes, seuil 12/30 ; périmètre (séances 1 à 6, ou 7) **À VÉRIFIER**
Rattrapage	dernière semaine du semestre (du 2026-12-07 au 2026-12-11), jour **À VÉRIFIER**
Examen final	période de session ; date, format, documents autorisés **À VÉRIFIER**
Note finale	pondération midterm et examen **À VÉRIFIER**
Présence	séance pratique du jeudi obligatoire : 70 %, 4 absences au plus

Source : fiche `CLAUDE.md` de la matière (annonce de l'enseignant du 2026-09-30) ; `hta1-intro.pdf`, p. 3

Selon l'enseignant (rapporté le 2026-09-30) : notation bienveillante, une explication approximative mais dans le bon sens rapporte des points, les questions vérifient la compréhension des notions.

Ce que les slides disent de l'examen. Les diapositives « Standardization Bodies » (`hta1p-protocols.pdf`, p. 9 à 11) et « ASN.1 Example » (p. 23) sont marquées « Not exam material ». La p. 26 annonce une question possible : « turn the given XML into JSON/YAML ». Sont aussi marquées « Not exam material » : « Sensor Networks » (`hta4-routing.pdf`, p. 17), « Jokes » (`hta5-l4.pdf`, p. 17) et « WWW Alternatives » (`hta6-applications.pdf`, p. 16). Ce document signale ces pages et ne les développe pas.

Usage. Un concept par page ; les concepts 6 à 17 (adressage IPv4), 30 et 31 (routage) et 41 à 43 (TCP) sont des méthodes de calcul, à refaire à la main. Concepts 28 à 33 : séance 4 (routage) ; 34 à 43 : séance 5 (transport, TCP) ; 44 à 51 : séance 6 (Web, e-mail). Exercices dans « NET, exercices corrigés ».

Sources. Chaque section cite sa slide (fichier, page du PDF). Les méthodes de calcul sont du savoir standard, signalé « méthode générale ». Une explication absente des slides, tirée d'une RFC ou d'un manuel, est signalée « rappel général ». Compléments cités comme tels : le TD 2019 du même cours (`HTA_GYAK_ALL_2019.pdf`, en hongrois). Tous les exemples chiffrés sont recalculés : adressage avec le module Python `ipaddress` (`calc_net.py`), séances 4 à 6 avec `calc_net_s46.py`. L'incertain est marqué **À VÉRIFIER**. Tableaux binaires : bleu pour le préfixe, ocre pour la partie hôte, | à la frontière.

Table des matières

1 Télécommunication, partage du support, adressage **8**

1.1 Partager un support commun (p. 16 à 21) 8

1.2 Modes de communication et adressage (p. 22 à 25) 9

2 Protocoles, pile, modèles OSI et TCP/IP **10**

2.1 Standardisation (p. 7 et 8) 11

3 Sécurité et qualité de service **12**

3.1 Sécurité (p. 12 à 15) 12

3.2 QoS (p. 16 à 19) 12

4 Multiplexage statistique **13**

5 Sérialisation : TLV, XML, JSON, YAML **14**

5.1 Méthode : convertir du XML en JSON puis en YAML (méthode générale) 15

6 Binaire, décimal, hexadécimal **16**

6.1 Décimal vers binaire, méthode des poids (soustractions) . . . 16

6.2 Décimal vers binaire, divisions successives par 2 16

6.3 Binaire vers décimal 17

6.4 Hexadécimal 17

7 Adresse IPv4 et notation pointée **19**

8 Classes historiques **20**

8.1 Pourquoi les classes ont été abandonnées (p. 5) 20

9 Adresses spéciales et privées **21**

10 Masque de sous-réseau et notation CIDR **22**

10.1 /n vers masque 22
10.2 Masque vers /n 22
10.3 Taille d'un réseau 23

11 Réseau, broadcast, plage d'hôtes : méthode du ET **24**

12 Méthode rapide du nombre magique **26**

13 Appartenance d'une adresse à un réseau **27**

14 Découpage en sous-réseaux de taille fixe **28**

15 VLSM : sous-réseaux de tailles différentes **30**

16 Agrégation de routes et supernetting **31**

17 Décision de routage : le plus long préfixe **33**
17.1 Lire une table `route -n` (capture de `hta3p` p. 12) 34

18 Couche réseau et modèle IPv4 **35**
18.1 Routage (p. 7 et 8) 35

19 En-tête IPv4 **36**

20 Fragmentation **38**
20.1 Méthode de calcul (méthode générale) 38

21 Plans et signalisation **39**

22 ICMP **40**
22.1 Ping et traceroute (p. 6) 41
22.2 Sécurité (p. 7) 41

23 DHCP **42**

23.1 Outils et démo (hta3p) 42

24 ARP **44**

25 DNS **45**
25.1 Résolution (p. 14 et 15) 45
25.2 Organisation, transport, sécurité (p. 16 à 18) 45

26 Outils de configuration réseau **46**
26.1 Configuration permanente (p. 15 et 16) 46

27 Outils de diagnostic et de capture **48**

28 Routage : rôle, tables, chemin optimal **49**
28.1 Chemin optimal (p. 5) 49
28.2 Sémantique du routage (p. 6) 50

29 Vecteur de distance et état des liens **51**
29.1 Étapes (hta4p, p. 4 et 7) 51
29.2 Pseudo-code des slides (hta4p, p. 3) 52

30 Méthode : Dijkstra pas à pas **53**

31 Méthode : vecteur de distance et count to infinity **55**
31.1 Exemple de la chaîne A-B-C-D-E-F (p. 5) 55
31.2 Count to infinity (p. 6) 55

32 RIP, OSPF, IS-IS et la démo BIRD **58**
32.1 OSPF en détail (p. 11) 58
32.2 Démo (hta4p, p. 15 à 17) 58

33 Topologie d'Internet, AS et BGP **60**
33.1 Relations entre AS (p. 14) 60

33.2 BGP (p. 15) 60

34 Couche transport et ports **62**
34.1 Contrôle de flux et de congestion (p. 5) 62
34.2 Datagrammes et flux (p. 6) 63

35 UDP **64**

36 TCP : connexion **65**
36.1 Sécurité (p. 10) 66

37 TCP : acquittement, fenêtre, en-tête **67**
37.1 Acquittement et fenêtre (p. 11) 67
37.2 En-tête (p. 13 à 16) 67

38 NAT **69**

39 Autres protocoles de transport **71**

40 Contrôle de flux et de congestion TCP **72**
40.1 Contrôle de flux (p. 3) 72
40.2 Congestion (p. 4 à 6) 72
40.3 Le contrôle de congestion TCP (p. 7) 73

41 Méthode : slow start et congestion avoidance **74**

42 TCP Reno, variantes et CUBIC **76**
42.1 Fast retransmit (p. 13) 76
42.2 Fast recovery (p. 14) 76
42.3 Variantes (p. 17 à 19) 77
42.4 Ce qu'il faut savoir (p. 20 et 21) 77

43 Produit bande passante délai, liens à pertes **78**

43.1 Liens à fort taux de perte (p. 17) 78

44 Le Web : navigateurs, serveurs, HTML, JavaScript **80**

45 HTTP et ses versions **81**

46 État HTTP : cookies **83**

47 HTTPS, clés publiques, TLS **84**
47.1 Publier les clés publiques (p. 13) 84
47.2 SSL et TLS (p. 14) 84

48 QUIC **86**

49 E-mail : architecture, format, stockage **87**
49.1 Architecture (p. 4) 87
49.2 Format (p. 5) 87
49.3 Stockage (p. 6) 88

50 SMTP, POP3, IMAP, clients et serveurs **89**

51 Sécurité de l'e-mail : authentification, PGP, spam, phishing **90**
51.1 Authentification du client (p. 13 et 14) 90
51.2 PGP (p. 15) 90
51.3 Spam (p. 16 à 18) 91
51.4 Phishing (p. 19) 91

52 Méthode d'examen **92**

53 Formulaire **95**

1. Télécommunication, partage du support, adressage

Source : `hta1-intro.pdf`, p. 4 à 25

La télécommunication, c'est partager de l'information à distance. Le cours retrace l'histoire (courrier, signaux optiques, télégraphe et code Morse, téléphone de Bell et Gray en 1876, standard de Tivadar Puskás en 1877, réseau téléphonique commuté PSTN, numéros E.164), puis passe au réseau informatique, sujet du cours. La téléphonie mobile est hors du cours (matière VIHIAC04, p. 14).

Calcul de la slide p. 13 : débit de la téléphonie numérique (PCM)

Voix limitée à 4 kHz (bande téléphonique 0,3 à 3,4 kHz). Nyquist : échantillonner au double de la fréquence maximale, donc 8 kHz. Chaque échantillon quantifié sur 8 bits (256 niveaux, quantification logarithmique). Débit : $8000 \times 8 = 64\,000$ bit/s, soit 64 kbit/s.

Partager un support commun (p. 16 à 21)

Problème	Définition et variantes
Multiplexage	une même source combine plusieurs flux sur un canal (exemple : voie descendante d'une antenne mobile). FDM par fréquence (modulation sur porteuse), TDM par tranches de temps (impossible en analogique), CDM par codes orthogonaux sur la même bande et au même moment
Accès multiple	plusieurs sources indépendantes se partagent le canal (voie montante des mobiles). FDMA, TDMA (détection de porteuse ou synchronisation), CDMA (codes non parfaitement orthogonaux sans synchronisation)
Duplexage	émettre dans les deux sens : FDD (deux fréquences), TDD (chacun son tour), ou un fil par sens (Ethernet, USB)
Sauts multiples	des stations intermédiaires relaient le trafic

Arbitrage du canal (p. 21) : en multiplexage, l'émetteur attribue les canaux ; en accès multiple, l'allocation est fixe (fréquences en cellulaire), gérée par une entité centrale sur demande (tranches de temps en cellulaire), ou dis-

putée (Ethernet CSMA).

Modes de communication et adressage (p. 22 à 25)

- Point à point : unicast, anycast. Point à multipoint : broadcast, multicast (dont geocast).
- Adresse de destination pour désigner le récepteur, adresse source pour la réponse.
- Unicast : identifiant unique par nœud. Multicast : adresse de groupe, rejoindre et quitter le groupe. Geocast : coordonnées géographiques, les nœuds doivent se localiser.
- Attribution : manuelle, par un nœud responsable, ou choisie par chaque nœud. Avec des liens point à point, on adresse plutôt les interfaces, et une interface peut avoir plusieurs adresses.
- Modèle centralisé (efficace, mais à installer, à croire et vulnérable) ou distribué (consensus moins efficace, plus robuste). Exemples de la slide : cellulaire contre Wi-Fi, serveur HTTP contre BitTorrent, Skype avant et après 2011.

2. Protocoles, pile, modèles OSI et TCP/IP

Source : `hta1p-protocols.pdf`, p. 2 à 8

Un protocole est un ensemble de règles (syntaxe, sémantique, algorithmes, automates, interfaces) qui permet à des entités d'échanger de l'information. On les empile en couches : chaque couche ne fait qu'une partie du travail et utilise la couche du dessous.

À retenir

- **SAP** (Service Access Point) : méthodes d'accès aux fonctions d'une couche, par exemple Connect(), Send(), Receive(). L'API réseau est typiquement les sockets BSD.
- **PDU** (Protocol Data Unit) : chaque couche dialogue « virtuellement » avec la même couche en face, mais ne parle réellement qu'aux couches voisines.
- **Encapsulation** : chaque couche ajoute son en-tête (parfois un pied) et ne touche qu'à ses propres en-têtes. Ordre sur le fil : en-tête 3, en-tête 2, en-tête 1, données.

OSI (ISO 7498)	PDU	TCP/IP	Rôle OSI
7 Application	Data	Application	utilisateur de la pile
6 Présentation			encodage, sérialisation, compression, chiffrement
5 Session			connexion, déconnexion, keepalive
4 Transport	Segment	Transport	échange fiable : segmentation, acquittement, retransmission, multiplexage
3 Réseau	Packet	Network	entre deux nœuds quelconques : relais, routage
2 Liaison	Frame	Link	vers un voisin (plusieurs voisins possibles)
1 Physique	Bit		bits sur le support

La slide p. 6 place la limite Application et Transport de TCP/IP au milieu

de la couche Session d'OSI. TCP/IP vient d'Unix BSD, définit des protocoles plutôt que des responsabilités de couche, et c'est lui qui est utilisé ; OSI est trop complexe et son implémentation n'existe que sur le papier. « Octet » plutôt que « byte » : le byte n'avait pas de taille fixe dans les années 1970.

Standardisation (p. 7 et 8)

Modèle boîte noire : on spécifie rigoureusement ce qui est visible de l'extérieur (interfaces, formats de PDU, séquences de messages), pas l'implémentation. Test de conformité : répondre aux entrées données par les sorties attendues ; entrées valides et invalides, couverture proche de 100 %, fuzzing.

> **Piège**
>
> Les p. 9 à 11 (IETF, IEEE, ITU, MPEG, ISO…) sont marquées « Not exam material ».

3. Sécurité et qualité de service

Source : `hta1p-protocols.pdf`, p. 12 à 19

Sécurité (p. 12 à 15)

- Se conçoit dès le départ ; l'ajouter après coup est difficile.
- **Chiffrement** contre l'écoute. Échange de clés : Diffie et Hellman (1976), secret partagé sur un canal non sûr grâce à l'arithmétique modulaire.
- **Authentification** : identifier la source (signature) ; contre le rejeu, défi et réponse avec un aléa cryptographique. AAA : Authentication, Authorization, Access control.
- **Intégrité** : une simple somme de contrôle se recalcule ; il faut y inclure un secret partagé (hachage type SHA).

QoS (p. 16 à 19)

Réserver des ressources suppose une entité qui « possède » le réseau. IntServ (RFC 1633) réserve par flux, DiffServ (RFC 2475) par classe ; best effort, sans réservation, est le régime d'Internet. Limites négociées (Call Admission Control), imposées à l'entrée du réseau (UNI). RED (Random Early Detection) jette avec une certaine probabilité avant que le seau soit plein ; TCP ralentit alors.

	Token bucket	Leaky bucket
Mécanisme	$+K$ jetons/s, capacité B ; un message coûte M_i ; seau vide : rejet	un message ajoute M_i ; sortie $\leq K$ jetons/s ; seau plein : rejet
Effet	mesure, passe ou échoue ; le débit peut dépasser K brièvement	mise en forme : sortie $\leq K$, il faut une file
Dimensionnement	K = débit moyen, $B = (\text{max} - \text{moyen}) \times \text{durée de rafale}$	

Exemple chiffré (valeurs choisies) : moyen 2 Mbit/s, max 10 Mbit/s, rafales de 0,5 s : $K = 2$ Mbit/s, $B = (10 - 2) \times 0{,}5 = 4$ Mbit.

4. Multiplexage statistique

Source : `hta1p-protocols.pdf`, p. 20

Question de la slide : $N = 500$ usagers, chacun actif avec probabilité $p = 1\,\%$. Combien de liens L faut-il ?

À retenir

- Moyenne : $Np = 5$ usagers actifs.
- Nombre d'actifs X : loi binomiale (N essais, succès p), approchée par une loi de Poisson de paramètre $\lambda = Np$ quand N est grand et p petit.
- Probabilité de blocage avec L liens : $\Pr(X > L) = 1 - \mathrm{CDF}(X, L)$.
- Inégalité de Bienaymé et Tchebychev : $\Pr(|X - \mu| \geq k\sigma) \leq 1/k^2$, vraie pour toute loi, donc elle surestime.
- Modèle plus réaliste pour la téléphonie : Erlang (probabilité d'appel, durée moyenne).

Piège

La slide écrit $\sigma = Np(1-p)$ pour la binomiale et $\sigma = \lambda$ pour Poisson. Ce sont les *variances*. Les écarts types sont $\sigma = \sqrt{Np(1-p)}$ et $\sigma = \sqrt{\lambda}$. La slide en déduit $L = \lambda + 3\lambda = 20$ et $\Pr(|X - 5| \geq 15) \leq 1/9$. Avec $\sigma = \sqrt{5} \approx 2{,}24$, on aurait $L = 5 + 3\sqrt{5} \approx 11{,}7$ et, pour $L = 20$, $k = 15/\sqrt{5} \approx 6{,}7$, soit une borne $1/k^2 = 1/45 \approx 0{,}022$. Quelle version l'enseignant attend à l'examen : **À VÉRIFIER**.

Valeurs exactes (calculées en Python, pour se situer)

Poisson, $\lambda = 5$: $\Pr(X > 20) \approx 8{,}1 \times 10^{-8}$; binomiale exacte : $\approx 6{,}3 \times 10^{-8}$. Avec 11 liens, Poisson donne $\Pr(X > 11) \approx 0{,}0055$. Tchebychev donne une borne très au-dessus de ces valeurs : c'est le prix de sa généralité.

5. Sérialisation : TLV, XML, JSON, YAML

Source : `hta1p-protocols.pdf`, p. 21 à 26

Couche présentation. Les machines ne représentent pas les données de la même façon : entiers 32 ou 64 bits, ordre des octets (Intel en little endian, le réseau en big endian, conversions `htons()`, `htonl()`, `ntohs()`, `ntohl()`), flottants. Il faut une représentation indépendante de la machine. Critères : lisible par un humain, auto-descriptive, extensible (sauter ce qu'on ne connaît pas), résistante aux erreurs, lisible à partir du milieu.

À retenir

- **TLV** (Type, Length, Value) : la longueur connue permet de sauter les éléments inconnus, donc une nouvelle version peut ajouter des types. Type composé : la valeur est une suite de TLV. DHCP code ses options en TLV (`hta3-ipv4-protocols.pdf`, p. 8).
- **ASN.1** (ITU-T, X.680) : langage de description d'interfaces, sérialisé en TLV. Règles BER, CER, DER (X.690, faciles à analyser), PER (X.691, compact). Utilisé par SS7, SNMP, la signalisation 2G à 5G (PER), les certificats X.509 (DER).
- **XML** : langage à balises issu de HTML ; validation par DTD ou XSD, XPath, XSLT, espaces de noms, API SAX. L'encodage XER d'ASN.1 est du XML (X.693).
- **JSON** : scalaires, tableaux `[]`, objets `{}` ; `JSON.parse()` plutôt que `eval()`. L'encodage JER d'ASN.1 est du JSON (X.697).
- **YAML** : hiérarchie par indentation, listes par tiret, dictionnaires `clé: valeur`. Sur-ensemble de JSON. Défauts : pas de validation de schéma, erreurs d'indentation, troncature indétectable (marqueurs de début et de fin optionnels). Surtout pour les fichiers de configuration.

Piège

L'exemple ASN.1 détaillé de la p. 23 (spécification et encodage DER octet par octet) est marqué « Not exam material ». La conversion XML vers JSON ou YAML, elle, est annoncée comme question possible (p. 26).

Méthode : convertir du XML en JSON puis en YAML (méthode générale)

1. L'élément racine englobe tout : son contenu devient l'objet JSON de premier niveau (la slide p. 25 omet le nom de la racine `NTALecture`).
2. Un élément qui ne contient que du texte devient une paire clé et valeur : `"seq": 2`. Un nombre peut s'écrire sans guillemets, le texte entre guillemets.
3. Un élément qui contient des éléments différents devient un objet `{}`.
4. Des éléments frères de même nom deviennent un tableau `[]` ; la balise répétée disparaît (la slide transforme les `SlideTitle` en simple liste).
5. YAML : mêmes données, on retire accolades, crochets et guillemets inutiles ; un niveau d'indentation par niveau d'imbrication ; chaque élément de liste commence par un tiret.
6. Les attributs XML n'ont pas d'équivalent imposé : choisir une convention et l'écrire (question non traitée par les slides).

Exemple de la slide (p. 24 à 26)

```
<NTALecture>
  <seq>2</seq>
  <title>Protocols Practical</title>
  <slides>
    <SlideTitle>Title</SlideTitle>
    <SlideTitle>Practical</SlideTitle>
    <SlideTitle>Buckets</SlideTitle>
    <SlideTitle>Multiplexing</SlideTitle>
  </slides>
</NTALecture>
```

JSON :

```
{
  "seq": 2,
  "title": "Protocols Practical",
  "slides": [ "Title",
    "Practical", "Buckets",
    "Multiplexing" ]
}
```

YAML :

```
seq: 2
title: Protocols Practical
slides:
  - Title
  - Practical
  - Buckets
  - Multiplexing
```

6. Binaire, décimal, hexadécimal

Source : méthode générale ; utilisée dans `hta2p-ipv4-practical.pdf`, p. 6 et 11 à 14

Une adresse IPv4 est une suite de 32 bits. Toutes les méthodes de calcul reposent sur trois bases : binaire (base 2), décimal (base 10), hexadécimal (base 16, chiffres 0 à 9 puis A à F).

À retenir

Puissances de 2 d'un octet, à savoir par cœur, de gauche à droite :

Rang du bit	7	6	5	4	3	2	1	0
Poids	128	64	32	16	8	4	2	1

Un octet vaut de 0 (`00000000`) à 255 (`11111111`). $2^{10} = 1024$, $2^{16} = 65\,536$, $2^{24} = 16\,777\,216$, $2^{32} = 4\,294\,967\,296$.

Décimal vers binaire, méthode des poids (soustractions)

Parcourir les poids de 128 à 1 : si le reste est supérieur ou égal au poids, écrire 1 et soustraire ; sinon écrire 0.

172 en binaire

Poids	128	64	32	16	8	4	2	1
Reste avant	172	44	44	12	12	4	0	0
Bit	1	0	1	0	1	1	0	0
Reste après	44	44	12	12	4	0	0	0

$172 = 128 + 32 + 8 + 4 =$ `10101100`. C'est le premier octet de 172.16.0.0, écrit `10101100` sur la slide p. 8 de `hta2p`.

Décimal vers binaire, divisions successives par 2

Diviser par 2 jusqu'à obtenir 0 ; les restes lus *de bas en haut* donnent le nombre binaire.

77 en binaire

Dividende		Quotient	Reste
77	÷2	38	1
38	÷2	19	0
19	÷2	9	1
9	÷2	4	1
4	÷2	2	0
2	÷2	1	0
1	÷2	0	1

Lecture de bas en haut : `1001101`, complété à 8 bits : `01001101`. Contrôle par les poids : $64 + 8 + 4 + 1 = 77$.

Binaire vers décimal

Additionner les poids des bits à 1.

`10110110` et `11100000`

$10110110 = 128+32+16+4+2 = 182$. $11100000 = 128+64+32 = 224$.

> **Piège**
>
> La slide `hta2p` p. 11 écrit « 224 is 1110000 » : il manque un zéro. Un octet a toujours 8 bits, 224 s'écrit `11100000`. Toujours compléter à 8 bits par des zéros à gauche.

Hexadécimal

Un chiffre hexadécimal code exactement 4 bits (un quartet), donc un octet s'écrit avec deux chiffres hexadécimaux.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Déc.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin.	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

(Lire le binaire de haut en bas : B = `1011`.)

- **Binaire vers hex** : couper en quartets à partir de la droite, traduire chacun. `1010 1100` → A, C : 172 = 0xAC.
- **Hex vers binaire** : chaque chiffre donne 4 bits. 0x45 → `0100 0101`.
- **Hex vers décimal** : $\text{chiffre}_1 \times 16 + \text{chiffre}_0$. 0xC8 = $12 \times 16 + 8 = 200$. Sur 16 bits : 0x6F89 = $6 \times 4096 + 15 \times 256 + 8 \times 16 + 9 = 28\,553$ (l'Identification de la capture Wireshark, `hta3p` p. 22).
- **Décimal vers hex** : divisions successives par 16, restes lus de bas en haut.

3000 en hexadécimal

$3000 = 187 \times 16 + 8$; $187 = 11 \times 16 + 11$; $11 = 0 \times 16 + 11$. Restes de bas en haut : 11, 11, 8, soit B, B, 8 : 3000 = 0xBB8. Contrôle : $11 \times 256 + 11 \times 16 + 8 = 2816 + 176 + 8 = 3000$.

7. Adresse IPv4 et notation pointée

Source : `hta2-ipv4.pdf`, p. 6 ; `hta2p-ipv4-practical.pdf`, p. 2

À retenir

- 32 bits, soit 4 octets. Notation décimale pointée : chaque octet écrit en décimal (0 à 255), séparés par des points. Exemple de la slide : 152.66.244.17.
- Deux parties : **préfixe réseau** (à gauche) et **identifiant d'hôte** (à droite). Où passe la frontière : voir classes (concept 8) puis CIDR (concept 10).
- L'adresse identifie une *interface*, pas un nœud ; une interface peut avoir plusieurs adresses.
- Les nœuds de même préfixe que moi sont joignables directement ; les autres, via la passerelle configurée.
- Routage en deux phases : atteindre le réseau cible grâce au préfixe, puis trouver l'hôte dans ce réseau.
- Modes : unicast (usage normal), multicast (groupe défini avec IGMP), broadcast (seulement dans le sous-réseau).

152.66.244.17 en binaire et en hexadécimal

Octet	152	66	244	17
Binaire	10011000	01000010	11110100	00010001
Hex	98	42	F4	11

$152 = 128+16+8$; $66 = 64+2$; $244 = 128+64+32+16+4$; $17 = 16+1$.
En hexadécimal sur 32 bits : 0x9842F411.

Piège

Un octet ne dépasse jamais 255. « 152.66.140.266 » n'est pas une adresse : la slide `hta2p` p. 12 le classe « invalid ».

8. Classes historiques

Source : `hta2p-ipv4-practical.pdf`, p. 3 à 5

Adressage d'origine (années 1980) : la position de la frontière se lit dans les premiers bits.

Classe	Premiers bits	Préfixe	1er octet	Usage
A	`0`	8 bits	1 à 126	unicast
B	`10`	16 bits	128 à 191	unicast
C	`110`	24 bits	192 à 223	unicast
D	`1110`		224 à 239	multicast
E	`11110`		240 et plus	réservé

Plages A, B, C : slide p. 4. D et E se déduisent des premiers bits (`11100000` = 224, `11101111` = 239) ; la slide ne donne pas la plage exacte de E.

Classe	Réseaux	Hôtes par réseau	Justification
A	126	16 777 214	$2^7 = 128$ moins 0 et 127 ; $2^{24} - 2$
B	16 384	65 534	2^{14} (16 bits moins 2 fixés) ; $2^{16} - 2$
C	2 097 152	254	2^{21} (24 bits moins 3 fixés) ; $2^8 - 2$

> **À retenir**
>
> - Le réseau 127 est réservé au loopback.
> - Partie hôte tout à 0 : « ce réseau entier » (adresse de réseau). Tout à 1 : broadcast. D'où le « −2 » dans chaque nombre d'hôtes.

Pourquoi les classes ont été abandonnées (p. 5)

Classe A trop grande : on la découpe par service (**subnetting**, trois niveaux : réseau, sous-réseau, hôte). Classe C trop petite : 2000 machines demanderaient 8 blocs C. Les petits blocs ajoutés multiplient les préfixes et les tables de routage explosent. Réponse : CIDR.

9. Adresses spéciales et privées

Source : `hta2p-ipv4-practical.pdf`, p. 4 et 8

Plage	Rôle
127.0.0.1	loopback (l'interface locale elle-même)
10.0.0.0/8	privée : un réseau de classe A
172.16.0.0/12	privée : 16 réseaux de classe B contigus (172.16 à 172.31), masque 255.240.0.0
192.168.0.0/16	privée : 256 réseaux de classe C contigus
hôte tout à 0	adresse du réseau
hôte tout à 1	broadcast du réseau

Les adresses privées ne sont pas routables sur Internet (on passe par du NAT) et on les découpe couramment en sous-réseaux plus petits.

Pourquoi 172.16.0.0/12 couvre 172.16 à 172.31

Slide p. 8 : `10101100.0001xxxx.y.z`. Les 12 premiers bits sont fixés ; les 4 bits `xxxx` du deuxième octet varient de `0000` à `1111`, donc le deuxième octet va de `00010000` = 16 à `00011111` = 31. 172.32.0.1 est hors de la plage : publique.

Piège

La slide écrit « 10.0.0.0/24 » pour « one Class A network » et « 127.0.0.0/24 » pour le loopback. Un réseau de classe A a un préfixe de 8 bits, et les RFC 1918 (privées) et 1122 (loopback) donnent 10.0.0.0/8 et 127.0.0.0/8. Coquille probable de la slide ; ce document écrit /8. Confirmation auprès de l'enseignant : **À VÉRIFIER**.

Deux adresses vues dans les slides, sans y être expliquées. 0.0.0.0 avec le masque 0.0.0.0 est la *route par défaut* dans la table `route -n` (`hta3p` p. 12). Le DHCP Discover part en « IP broadcast » (`hta3` p. 10) : l'adresse utilisée est 255.255.255.255, broadcast limité (RFC 2131, hors slides).

10. Masque de sous-réseau et notation CIDR

Source : `hta2p-ipv4-practical.pdf`, p. 6, 7 et 11

CIDR (Classless Inter-Domain Routing, 1992) : la frontière réseau et hôte peut être n'importe où. Deux notations équivalentes : `a.b.c.d/n` (longueur de préfixe n) ou un masque, n bits à 1 suivis de $32 - n$ bits à 0. Exemple de la slide : $n = 14$ donne 255.252.0.0, car 252 = `11111100`.

Masque /14 `255.252.0.0` `11111111.111111`|`00.00000000.00000000`

À retenir

Valeurs possibles d'un octet de masque (nombre de 1 en tête) :

Bits à 1	0	1	2	3	4	5	6	7	8
Octet	0	128	192	224	240	248	252	254	255

Chaque valeur ajoute le poids suivant : 128, +64 = 192, +32 = 224, +16 = 240…

/n vers masque

Écrire $n = 8 + 8 + \cdots + r$ avec $r < 8$: autant d'octets 255 que de 8, puis l'octet qui a r bits à 1, puis des 0.

/19 et /13 (le premier est l'exemple 1 de la slide p. 11)

19 = 8 + 8 + 3 : 255.255, puis 3 bits à 1 = 224, puis 0 : **255.255.224.0**.
13 = 8 + 5 : 255, puis 5 bits à 1 = 248 : **255.248.0.0**.

Masque vers /n

Compter les 1 : 8 par octet 255, plus les bits de l'octet partiel (table ci-dessus).

255.224.0.0 (slide p. 11) et 255.255.255.240

255.224.0.0 : 8 + 3 (224 = `11100000`) = **/11**.
255.255.255.240 : 8 + 8 + 8 + 4 = **/28**.

Piège

Un masque doit être une suite de 1 *puis* une suite de 0. 255.96.0.0 est invalide (slide p. 11) : 96 = `01100000`, un 0 précède des 1. De même 255.255.255.160 (160 = `10100000`) ou 255.0.255.0.

Taille d'un réseau

Un préfixe /n laisse $h = 32 - n$ bits d'hôte : 2^h adresses, dont $2^h - 2$ attribuables (on retire réseau et broadcast). C'est le tableau de la slide p. 7, prolongé ci-dessous.

/n	Masque	Bloc	Hôtes	/n	Masque	Bloc	Hôtes
/8	255.0.0.0	2^{24}	16 777 214	/24	255.255.255.0	256	254
/16	255.255.0.0	2^{16}	65 534	/25	255.255.255.128	128	126
/17	255.255.128.0	2^{15}	32 766	/26	255.255.255.192	64	62
/18	255.255.192.0	2^{14}	16 382	/27	255.255.255.224	32	30
/19	255.255.224.0	2^{13}	8 190	/28	255.255.255.240	16	14
/20	255.255.240.0	2^{12}	4 094	/29	255.255.255.248	8	6
/21	255.255.248.0	2^{11}	2 046	/30	255.255.255.252	4	2
/22	255.255.252.0	1024	1 022				
/23	255.255.254.0	512	510				

Plus petit préfixe pour H hôtes (méthode générale) : chercher le plus petit h tel que $2^h - 2 \geq H$, puis $n = 32 - h$. Pour 300 hôtes : $2^8 - 2 = 254$ ne suffit pas, $2^9 - 2 = 510$ suffit, donc /23. Pour 62 hôtes : /26 ($62 \geq 62$). Pour 63 : /25.

Piège

Les préfixes /31 et /32 ne suivent pas la règle « −2 »; les slides n'en parlent pas. La table `route -n` de `hta3p` p. 12 montre des routes vers un seul hôte, masque 255.255.255.255 (drapeau H).

11. Réseau, broadcast, plage d'hôtes : méthode du ET

Source : méthode générale ; alignement binaire comme sur `hta2p-ipv4-practical.pdf`, p. 12 à 14

À retenir

1. Écrire l'adresse et le masque en binaire, alignés.
2. **Adresse réseau** = adresse ET masque, bit à bit : on garde le préfixe, on met la partie hôte à 0.
3. **Broadcast** = préfixe inchangé, partie hôte tout à 1.
4. **Première adresse d'hôte** = réseau +1 ; **dernière** = broadcast −1.
5. **Nombre d'hôtes** = $2^{32-n} - 2$.

ET logique : $1 \wedge 1 = 1$, sinon 0. Seul l'octet où le masque n'est ni 255 ni 0 demande un calcul ; ailleurs on recopie (masque 255) ou on met 0 (masque 0).

Exemple 1 : 192.168.10.77/26

Adresse	`192.168.10.77`	`11000000.10101000.00001010.01|001101`
Masque /26	`255.255.255.192`	`11111111.11111111.11111111.11|000000`
Réseau (ET)	`192.168.10.64`	`11000000.10101000.00001010.01|000000`
Broadcast	`192.168.10.127`	`11000000.10101000.00001010.01|111111`

Octet intéressant : le 4e. 77 = 01001101, masque 192 = 11000000. ET : 01000000 = 64. Broadcast : 01111111 = 127.
Réseau **192.168.10.64**, broadcast **192.168.10.127**, hôtes de 192.168.10.65 à 192.168.10.126, soit $2^6 - 2 = 62$ hôtes.

Exemple 2 : 152.66.163.9/18 (adresse de la slide p. 12)

Adresse	`152.66.163.9`	`10011000.01000010.10|100011.00001001`
Masque /18	`255.255.192.0`	`11111111.11111111.11|000000.00000000`
Réseau (ET)	`152.66.128.0`	`10011000.01000010.10|000000.00000000`
Broadcast	`152.66.191.255`	`10011000.01000010.10|111111.11111111`

Masque 255.255.192.0. Octet intéressant : le 3e. 163 = 10100011, 192 = 11000000, ET : 10000000 = 128. Broadcast : 10111111.11111111, soit

191.255.
Réseau **152.66.128.0**, broadcast **152.66.191.255**, hôtes de 152.66.128.1 à 152.66.191.254, soit $2^{14} - 2 = 16\,382$ hôtes.

Exemple 3 : 10.45.200.13/12

Adresse	10.45.200.13	00001010.0010|1101.11001000.00001101
Masque /12	255.240.0.0	11111111.1111|0000.00000000.00000000
Réseau (ET)	10.32.0.0	00001010.0010|0000.00000000.00000000
Broadcast	10.47.255.255	00001010.0010|1111.11111111.11111111

Masque 255.240.0.0. Octet intéressant : le 2e. 45 = 00101101, 240 = 11110000, ET : 00100000 = 32. Les octets 3 et 4 passent à 0 (réseau) ou 255 (broadcast). Broadcast du 2e octet : 00101111 = 47.
Réseau **10.32.0.0**, broadcast **10.47.255.255**, hôtes de 10.32.0.1 à 10.47.255.254, soit $2^{20} - 2 = 1\,048\,574$ hôtes.

Exemple 4 : 172.20.37.200/21

Adresse	172.20.37.200	10101100.00010100.00100|101.11001000
Masque /21	255.255.248.0	11111111.11111111.11111|000.00000000
Réseau (ET)	172.20.32.0	10101100.00010100.00100|000.00000000
Broadcast	172.20.39.255	10101100.00010100.00100|111.11111111

Masque 255.255.248.0. 37 = 00100101, 248 = 11111000, ET : 00100000 = 32. Broadcast : 00100111 = 39, puis 255.
Réseau **172.20.32.0**, broadcast **172.20.39.255**, hôtes de 172.20.32.1 à 172.20.39.254, soit $2^{11} - 2 = 2046$ hôtes.

Exemple 5 : 200.1.1.130/30, lien point à point

Adresse	200.1.1.130	11001000.00000001.00000001.100000|10
Masque /30	255.255.255.252	11111111.11111111.11111111.111111|00
Réseau (ET)	200.1.1.128	11001000.00000001.00000001.100000|00
Broadcast	200.1.1.131	11001000.00000001.00000001.100000|11

130 = 10000010, 252 = 11111100, ET : 10000000 = 128. Réseau **200.1.1.128**, broadcast **200.1.1.131**, deux hôtes : 200.1.1.129 et 200.1.1.130.

12. Méthode rapide du nombre magique

Source : méthode générale, équivalente à la méthode du ET

Pour calculer vite sans écrire 32 bits.

À retenir

1. Repérer l'**octet intéressant** : celui où le masque n'est ni 255 ni 0.
2. **Nombre magique** $m = 256 -$ octet du masque (taille du bloc).
3. Les réseaux commencent aux multiples du nombre magique : 0, m, $2m$…
4. Réseau : le plus grand multiple de m inférieur ou égal à la valeur de l'octet ; octets à droite à 0.
5. Broadcast : réseau $+m - 1$ dans l'octet intéressant ; octets à droite à 255.

Les exemples de la page précédente, en une ligne chacun

192.168.10.77/26	4e octet, $m = 256 - 192 = 64$. Multiples : 0, 64, 128. $64 \leq 77 < 128$: réseau .64, broadcast $64 + 63 = 127$.
152.66.163.9/18	3e octet, $m = 256 - 192 = 64$. $128 \leq 163 < 192$: réseau 152.66.128.0, broadcast 152.66.191.255.
10.45.200.13/12	2e octet, $m = 256 - 240 = 16$. Multiples 32, 48 : $32 \leq 45 < 48$: réseau 10.32.0.0, broadcast 10.47.255.255.
172.20.37.200/21	3e octet, $m = 256 - 248 = 8$. $32 \leq 37 < 40$: réseau 172.20.32.0, broadcast 172.20.39.255.

Piège

- Préfixe multiple de 8 (/8, /16, /24) : pas d'octet intéressant, octets hôte à 0.
- Un octet à droite de l'octet intéressant vaut 0 pour le réseau et 255 pour le broadcast, pas « $m - 1$ ».
- Une adresse de réseau ou de broadcast ne s'attribue pas : 193.225.14.63/28 est le broadcast de 193.225.14.48/28 ($48 + 15 = 63$).

13. Appartenance d'une adresse à un réseau

Source : `hta2p-ipv4-practical.pdf`, p. 12 (exemple 2)

À retenir

Deux adresses sont dans le même sous-réseau si et seulement si leurs n premiers bits sont identiques, autrement dit si (adresse 1 ET masque) = (adresse 2 ET masque). Méthode rapide : calculer la plage du réseau de référence et vérifier que l'adresse y tombe.

Exemple 2 de la slide : lesquelles sont sur le même sous-réseau que 152.66.163.9/18 ?

Nous	152.66.163.9	10011000.01000010.10|100011.00001001
(1)	152.66.178.180	10011000.01000010.10|110010.10110100
(2)	162.66.131.11	10100010.01000010.10|000011.00001011
(3)	152.66.195.210	10011000.01000010.11|000011.11010010
(4)	152.66.129.51	10011000.01000010.10|000001.00110011
(5)	152.66.140.266	10011000.01000010.10|001100.XXXXXXXX

152.66 est commun à toutes sauf (2) ; on compare les 2 premiers bits du 3e octet, 10 pour la référence.

- (1) 152.66.178.180 : 178 = 10110010, commence par 10 : **oui**.
- (2) 162.66.131.11 : le premier octet diffère (162 et non 152) : **non**.
- (3) 152.66.195.210 : 195 = 11000011, commence par 11 : **non** (réseau 152.66.192.0/18).
- (4) 152.66.129.51 : 129 = 10000001 : **oui**.
- (5) 152.66.140.266 : 266 dépasse 255, adresse **invalide**.

Réponse de la slide : 1 et 4. Méthode rapide : plage 152.66.128.0 à 152.66.191.255, 178 et 129 y sont, 195 non.

Piège

192.168.5.130/25 et 192.168.5.100/25 sont dans deux réseaux (.128/25 et .0/25) : ils passent par la passerelle. En /24, ils se joindraient directement.

14. Découpage en sous-réseaux de taille fixe

Source : principe : `hta2p-ipv4-practical.pdf`, p. 5 et 8 ; méthode générale

Subnetting : on emprunte les premiers bits de la partie hôte pour numéroter des sous-réseaux.

À retenir

- Emprunter s bits donne 2^s sous-réseaux de préfixe $n + s$.
- Pour S sous-réseaux : plus petit s tel que $2^s \geq S$.
- Pour au moins H hôtes par sous-réseau : plus petit h tel que $2^h - 2 \geq H$, puis préfixe $32 - h$.
- Sous-réseau k (compté depuis 0) : base + $k \times 2^{32-(n+s)}$.

192.168.20.0/24 en au moins 6 sous-réseaux égaux

$4 < 6 \leq 8 = 2^3$: $s = 3$ bits, /27, 255.255.255.224, pas de 32, 30 hôtes chacun.

Réseau /24	192.168.20.0	11000000.10101000.00010100.|00000000
Masque /27	255.255.255.224	11111111.11111111.11111111.111|00000
Sous-rés. 0	192.168.20.0	11000000.10101000.00010100.000|00000
Sous-rés. 1	192.168.20.32	11000000.10101000.00010100.001|00000
Sous-rés. 2	192.168.20.64	11000000.10101000.00010100.010|00000

k	Réseau /27	Bcast	k	Réseau /27	Bcast
0	192.168.20.0	.31	4	192.168.20.128	.159
1	192.168.20.32	.63	5	192.168.20.160	.191
2	192.168.20.64	.95	6	192.168.20.192	.223
3	192.168.20.96	.127	7	192.168.20.224	.255

Hôtes : de réseau +1 à broadcast −1 (.33 à .62 pour $k = 1$). Deux restent libres.

172.16.0.0/16 en sous-réseaux d'au moins 500 hôtes

$254 < 500 \leq 510 = 2^9 - 2$: préfixe /23 (255.255.254.0), 7 bits emprun-

tés, $2^7 = 128$ sous-réseaux de 512 adresses, pas de 2 dans le 3e octet : 172.16.0.0, 172.16.2.0…172.16.254.0/23.
Sous-réseau $k = 36$: $36 \times 2 = 72$, **172.16.72.0/23**, hôtes 172.16.72.1 à 172.16.73.254, broadcast 172.16.73.255.

Base /16	`172.16.0.0`	`10101100.00010000.`|`00000000.00000000`
Masque /23	`255.255.254.0`	`11111111.11111111.1111111`|`0.00000000`
$k = 36$	`172.16.72.0`	`10101100.00010000.0100100`|`0.00000000`
Broadcast	`172.16.73.255`	`10101100.00010000.0100100`|`1.11111111`

Vérification en binaire : les 7 bits empruntés (bits 17 à 23) valent `0100100`, soit $32 + 4 = 36$; le dernier bit du 3e octet et tout le 4e octet forment la partie hôte (9 bits), d'où 72.0 pour le réseau et 73.255 pour le broadcast.

15. VLSM : sous-réseaux de tailles différentes

Source : méthode générale ; les slides disent seulement qu'avec CIDR « we can have a network of any size » (`hta2p`, p. 6)

VLSM (Variable Length Subnet Masking) : chaque sous-réseau reçoit le préfixe adapté à son besoin, pour ne pas gaspiller d'adresses.

À retenir

1. Classer les besoins du plus grand au plus petit.
2. Pour chacun, prendre le plus petit bloc 2^h avec $2^h - 2 \geq$ besoin.
3. Allouer à la suite, en partant de l'adresse de base. En commençant par les plus gros blocs, chaque bloc tombe automatiquement sur un multiple de sa taille (alignement).
4. Vérifier que le total ne dépasse pas le bloc de départ et qu'aucun bloc ne chevauche un autre.

192.168.50.0/24 : LAN de 100, 50, 20 et 10 hôtes, deux liens point à point

Tailles : 100 → 128 (/25), 50 → 64 (/26), 20 → 32 (/27), 10 → 16 (/28), 2 → 4 (/30) deux fois. Total $128 + 64 + 32 + 16 + 4 + 4 = 248 \leq 256$: ça tient.

Besoin	Réseau	Hôtes utilisables	Broadcast	Hôtes
100	192.168.50.0/25	.1 à .126	.127	126
50	192.168.50.128/26	.129 à .190	.191	62
20	192.168.50.192/27	.193 à .222	.223	30
10	192.168.50.224/28	.225 à .238	.239	14
2	192.168.50.240/30	.241 et .242	.243	2
2	192.168.50.244/30	.245 et .246	.247	2

Reste libre : 192.168.50.248/29 (de .248 à .255).

Piège

Allouer dans le désordre casse l'alignement : un /25 ne commence qu'à .0 ou .128 ; un /30 placé d'abord en .0 repousse le /25 à .128. Si la somme des blocs dépasse l'espace, le plan est impossible : le dire.

16. Agrégation de routes et supernetting

Source : `hta2p-ipv4-practical.pdf`, p. 6 et 14 (exemple 4)

Moins de lignes dans la table de routage, c'est une recherche plus rapide (slide p. 14).

> **À retenir**
>
> Deux préfixes de même longueur n se fusionnent en un préfixe $n-1$ si :
>
> - seul le dernier bit de leurs préfixes diffère (ils sont adjacents *et* alignés) ;
> - ils ont le même saut suivant (même interface).
>
> On répète tant que c'est possible. Exemple de la slide p. 6 : deux /18 adjacents donnent un /17 « si les 17 premiers bits sont identiques et le saut suivant aussi ».

Exemple 4 de la slide : quatre routes vers eth5

```
(1) /10  130.120.0.0  10000010.01|111000.00000000.00000000
(2) /10  130.128.0.0  10000010.10|000000.00000000.00000000
(3) /11  130.192.0.0  10000010.110|00000.00000000.00000000
(4) /11  130.224.0.0  10000010.111|00000.00000000.00000000
```

- (3) et (4) : préfixes de 11 bits, seul le 11e bit diffère (`0` et `1`). Plages 130.192.0.0 à 130.223.255.255 et 130.224.0.0 à 130.255.255.255, contiguës : fusion en **130.192.0.0/10**.
- (2) 130.128.0.0/10 et ce 130.192.0.0/10 : seul le 10e bit diffère : fusion en **130.128.0.0/9**.
- (1) « 130.120.0.0/10 » est mal écrit : 120 = `01111000`, les 2 premiers bits du 2e octet valent `01`, donc le vrai réseau est 130.64.0.0/10 (de 130.64.0.0 à 130.127.255.255). Avec 130.128.0.0/10, les deux derniers bits du préfixe diffèrent (`01` et `10`) : pas de fusion.

Résultat de la slide : (2), (3) et (4) deviennent 130.128.0.0/9 ; la table passe de 4 à 2 lignes.

Quatre /24 consécutifs

```
/24          192.168.4.0 11000000.10101000.000001|00.00000000
/24          192.168.5.0 11000000.10101000.000001|01.00000000
/24          192.168.6.0 11000000.10101000.000001|10.00000000
/24          192.168.7.0 11000000.10101000.000001|11.00000000
-------------------------------------------------------------
Résumé /22   192.168.4.0 11000000.10101000.000001|00.00000000
```

Les 22 premiers bits sont communs, les 2 bits suivants prennent les 4 valeurs : le résumé exact est **192.168.4.0/22**.

Deux /24 adjacents mais non alignés

```
/24 192.168.1.0 11000000.10101000.000000|01.00000000
/24 192.168.2.0 11000000.10101000.000000|10.00000000
```

192.168.1.0/24 et 192.168.2.0/24 sont contigus, mais leurs 3es octets `00000001` et `00000010` diffèrent sur les deux derniers bits : pas de /23 commun. Le plus petit préfixe qui les couvre, 192.168.0.0/22, contient aussi 192.168.0.0/24 et 192.168.3.0/24 : ce n'est plus une agrégation exacte.

17. Décision de routage : le plus long préfixe

Source : `hta2p-ipv4-practical.pdf`, p. 13 (exemple 3) ; `hta3p-ipv4-tools.pdf`, p. 12

À retenir

1. Pour chaque ligne de la table : la destination appartient-elle au préfixe ? (ET avec le masque de la ligne, comparer.)
2. Parmi les lignes qui correspondent, garder celle de **préfixe le plus long** (longest prefix match) : c'est la plus spécifique.
3. La route par défaut 0.0.0.0/0 correspond à tout, avec la longueur 0 : elle ne sert que si rien d'autre ne correspond.

Exemple 3 de la slide : destination 170.85.167.2

Dest.	`170.85.167.2`	`10101010.010101`|`01.10100111.00000010`
/13 eth7	`170.80.0.0`	`10101010.01010`|`000.00000000.00000000`
/13 eth2	`170.88.0.0`	`10101010.01011`|`000.00000000.00000000`
/14 eth11	`170.84.0.0`	`10101010.010101`|`00.00000000.00000000`

85 = `01010101`. Comparer les bits du 2e octet sur la longueur de chaque préfixe :

- 170.80.0.0/13 (eth7) : 5 bits du 2e octet, `01010` contre `01010` : correspond.
- 170.88.0.0/13 (eth2) : `01011` contre `01010` : non.
- 170.84.0.0/14 (eth11) : 6 bits, `010101` contre `010101` : correspond.

Lignes 1 et 3 correspondent ; /14 est plus long que /13 : sortie **eth11**.

Table avec route par défaut (exemple construit pour l'entraînement)

Table : 0.0.0.0/0 eth0 ; 10.0.0.0/8 eth1 ; 10.1.0.0/16 eth2 ; 10.1.4.0/22 eth3 ; 10.1.5.128/25 eth4.

Dest.	`10.1.5.20`	`00001010.00000001.00000101.0`|`0010100`
/8 eth1	`10.0.0.0`	`00001010.`|`00000000.00000000.00000000`
/16 eth2	`10.1.0.0`	`00001010.00000001.`|`00000000.00000000`
/22 eth3	`10.1.4.0`	`00001010.00000001.000001`|`00.00000000`
/25 eth4	`10.1.5.128`	`00001010.00000001.00000101.1`|`0000000`

- 10.1.5.20 : /8, /16 et /22 (plage 10.1.4.0 à 10.1.7.255) correspondent ; /25 couvre .128 à .255 du 10.1.5, et 20 n'y est pas. Sortie **eth3**.
- 10.1.5.200 : le /25 correspond aussi : **eth4**.
- 10.1.8.1 : le /22 couvre 10.1.4.x à 10.1.7.x, 8 en sort : **eth2**.
- 10.2.0.1 : seul /8 : **eth1**. 8.8.8.8 : route par défaut, **eth0**.

Lire une table `route -n` (capture de `hta3p` p. 12)

```
Destination     Gateway         Genmask          Flags Iface
0.0.0.0         192.168.100.254 0.0.0.0          UG    enp8s0f0
8.8.8.8         192.168.100.254 255.255.255.255  UGH   enp8s0f0
10.1.33.0       0.0.0.0         255.255.255.0    U     enp3s0
172.17.0.0      0.0.0.0         255.255.0.0      U     docker0
192.168.100.0   0.0.0.0         255.255.255.0    U     enp8s0f0
```

(extrait ; colonnes Metric, Ref, Use omises). Gateway 0.0.0.0 : réseau directement connecté. Drapeaux, selon la page de manuel de `route` (non expliqués sur la slide) : U route active, G passe par une passerelle, H destination d'un seul hôte. Un paquet vers 10.1.33.2 sort par enp3s0 sans passerelle ; vers 9.9.9.9, seule la route par défaut correspond : il part vers 192.168.100.254.

18. Couche réseau et modèle IPv4

Source : `hta2-ipv4.pdf`, p. 2 à 9 et 17

À retenir

- Couche réseau : échange de paquets entre deux nœuds quelconques. Adressage (identifiant unique), routage (trouver un chemin), relayage : {DST, SRC, saut précédent} → saut suivant.
- Commutation de circuits : circuit virtuel à établir avant d'envoyer. Commutation de paquets : chaque paquet routé individuellement.
- IPv4 : RFC 760 (1980), norme actuelle RFC 791 (1981) ; v0 à v3 n'étaient que des brouillons. Sans connexion, **best effort**, ni fiabilité ni ordre garantis ; supporte DiffServ.
- Sablier : tout le monde parle IP ; liaison et couches hautes restent locales.
- Terminal (ne relaie pas), routeur (relaie), passerelle (entre deux domaines).

Routage (p. 7 et 8)

Routage : découvrir la topologie (protocole de routage). Relayage : utiliser cette information pour choisir le saut suivant, paquet par paquet, chaque nœud de façon indépendante (hop by hop). Routage par la source (elle impose le chemin) : pas sur Internet, on ne peut pas lui faire confiance ; utilisé en réseaux de capteurs ad hoc.

Boucles : si l'information de topologie est fausse, un paquet peut tourner indéfiniment. Les nœuds ne mémorisent pas les paquets vus et les paquets ne stockent pas les nœuds visités. Solution : le **TTL**, compteur décrémenté à chaque nœud ; le paquet est détruit à 0.

Succès d'IPv4 (p. 17) : IPX et X.25 éliminés, remplacement global impossible ; IPv6 naît de l'épuisement des adresses.

Mobilité (`hta2p` p. 10) : l'adresse dépend de l'emplacement. Mobile IP : adresse « home » officielle, adresse locale dans le réseau visité, joignable à l'adresse home. **Allocation** (p. 9) : trop tardif, CIDR n'a pas rendu l'allocation géographique.

19. En-tête IPv4

Source : `hta2-ipv4.pdf`, p. 10 à 16

Bits	Champ	À retenir
4	Version	toujours 4
4	IHL	longueur d'en-tête en mots de 32 bits ; minimum 5 (20 octets)
6 + 2	DSCP, ECN	DSCP surtout en réseau privé ; ECN signale la congestion sans jeter, peu utilisé
16	Total Length	octets, en-tête compris : de 20 à 65 535
16	Identification	numéro du paquet, pour regrouper ses fragments
3	Flags	bit réservé, DF (ne pas fragmenter, jeter), MF (d'autres fragments suivent)
13	Fragment Offset	position dans le paquet d'origine, en unités de 8 octets
8	TTL	anti-boucle
8	Protocol	contenu : ICMP 1, IGMP 2, TCP 6, UDP 17 (liste IANA)
16	Header Checksum	en-tête seul ; inclut le TTL, donc recalculé par chaque routeur
32 + 32	Source, Destination	adresses IP
var.	Options	si IHL > 5 ; expérimentales, presque toutes dépréciées (RFC 6814)

0x45, premier octet de presque tous les paquets

0x45 = `0100 0101` : version `0100` = 4, IHL `0101` = 5, soit $5 \times 4 = 20$ octets d'en-tête, sans options. La slide p. 10 note que certaines implémentations codent 0x45 en dur.

Décoder l'en-tête de la capture Wireshark (`hta3p-ipv4-tools.pdf`, p. 22)

Les 20 octets IP, après les 14 octets Ethernet :

```
45 00 00 3c 6f 89 00 00 02 11 30 8d
c0 a8 64 0f ac 43 47 a0
```

`45`	version 4, IHL 5	20 octets
`00`	DSCP 0, ECN 0	
`00 3c`	Total Length	$3 \times 16 + 12 = 60$ octets
`6f 89`	Identification	28 553
`00 00`	Flags, Offset	DF = 0, MF = 0, offset 0
`02`	TTL	2
`11`	Protocol	$16 + 1 = 17$: UDP
`30 8d`	Checksum	0x308D
`c0 a8 64 0f`	Source	192.168.100.15
`ac 43 47 a0`	Destination	172.67.71.160

Un TTL de 2 et des ports UDP élevés : c'est un paquet de `traceroute` (les lignes voisines de la capture sont des « Time-to-live exceeded »). Wireshark affiche les mêmes valeurs.

Piège

IHL compte des mots de 4 octets, Total Length des octets, Fragment Offset des blocs de 8 octets. Trois unités différentes dans le même en-tête.

20. Fragmentation

Source : `hta2-ipv4.pdf`, p. 9 et 12 ; calcul : méthode générale

À retenir

- MTU : Ethernet 1500 octets, jumbo 9000 (non standard).
- On fragmente si le message dépasse la MTU ou si un lien du chemin a une MTU plus petite ; réassemblage au saut suivant ou au récepteur (question ouverte de la slide).
- Path MTU discovery (tester la MTU du chemin avant d'envoyer) : pas dans IPv4, selon la slide p. 9.
- DF à 1 : le routeur jette au lieu de fragmenter, et ICMP peut signaler « fragmentation needed » (type 3, `hta3` p. 4).

Méthode de calcul (méthode générale)

1. Données du paquet d'origine : D = Total Length − 20 (sans options).
2. Données par fragment : $\lfloor (\text{MTU} - 20)/8 \rfloor \times 8$, car l'offset compte en blocs de 8 octets ; seul le dernier fragment peut avoir une taille quelconque.
3. Offset d'un fragment = octets de données déjà envoyés /8.
4. MF = 1 sur tous les fragments sauf le dernier ; même Identification pour tous.

Paquet de 4000 octets (20 d'en-tête) sur un lien de MTU 1500

$D = 3980$. Par fragment : $1500 - 20 = 1480$, multiple de 8 (185×8).

Fragment	Données	Total Length	Offset	MF
1	1480	1500	0	1
2	1480	1500	185	1
3	1020	1040	370	0

Contrôle : 1480+1480+1020 = 3980 ; offset 185 = 1480/8, 370 = 2960/8.

Place de ce calcul à l'examen : les slides ne donnent pas d'exercice de fragmentation, **À VÉRIFIER**.

21. Plans et signalisation

Source : `hta3-ipv4-protocols.pdf`, p. 2 et 3

À retenir

- **Plan de données** : transporter les données utilisateur.
- **Plan de contrôle** : rendre le plan de données possible (établir et fermer une connexion, allouer des ressources QoS, acquitter, keepalive).
- **Plan de gestion** : rendre les deux autres possibles (routage, détection d'erreurs de lien, configuration). Définition jugée dépassée par la slide : aujourd'hui le routage est classé contrôle, la gestion se limite à la configuration à distance.
- ARP et DHCP sont « entre les couches » ; la slide laisse la question du plan ouverte.

Couche	Données	Contrôle	Gestion
Application	HTTP, RTP, DNS	RTCP, SIP	SNMP, OpenFlow
Transport	TCP, UDP	TCP	
Réseau	IPv4	ICMP	ICMP, RIP, OSPF
Liaison	Ethernet	LLC	STP

Signalisation : échanges de contrôle et de gestion qui ne sont pas des données utilisateur. *Dans la bande* : même canal que les données (« bonjour », « au revoir »). *Hors bande* : canal séparé (un signe de la main). L'ITU-T utilise souvent un canal dédié (fréquence ou tranche de temps) ; l'IETF envoie simplement des paquets IP.

22. ICMP

Source : `hta3-ipv4-protocols.pdf`, p. 4 à 7

À retenir

- Internet Control Message Protocol, RFC 792. Messages d'erreur (acquittements négatifs pour des conditions précises) et requêtes d'information, car IP est sans connexion et best effort.
- Encapsulé directement dans IP, **Protocol = 1**. Dans la bande pour IP, hors bande pour TCP et UDP.
- En-tête : **Type** (8 bits) et **Code** (8 bits, sous-type), plus une somme de contrôle, car IP ne contrôle pas sa charge utile.

Type	Message	Sous-types, usage
0	Echo Reply	réponse au ping
3	Destination Unreachable	net, host, protocol, port unreachable ; fragmentation needed ; net, host unreachable for ToS
5	Redirect	un routeur signale une meilleure route à la source (configuration manuelle)
8	Echo Request	ping
9, 10	Router Advertisement, Solicitation	découverte des routeurs locaux ; remplacé par DHCP, étendu par IPv6 Neighbor Discovery
11	Time Exceeded	TTL expiré ; délai de réassemblage dépassé

Piège

Ordre des types 9 et 10 : la slide écrit « Router Solicitation / Router Advertisement (type=9, 10) ». Selon la RFC 1256, 9 est l'Advertisement et 10 la Solicitation ; le tableau suit la RFC. Point à confirmer : **À VÉRIFIER**.

Ping et traceroute (p. 6)

Ping : Echo Request, attendre l'Echo Reply ; mesure latence et taux de perte. **Traceroute** : envoyer avec TTL = 1, 2, 3… ; le routeur où le TTL tombe à 0 renvoie un Time Exceeded, ce qui révèle chaque saut.

Sécurité (p. 7)

- Aucune authentification : qui intercepte mon trafic peut m'envoyer un faux Destination Unreachable ou Redirect.
- Déni de service : ping flood, saturation du tampon de réassemblage.
- Les messages d'erreur renvoient le paquet fautif (au moins 8 octets, jusqu'à la MTU) ; Echo peut transporter n'importe quelle quantité de données, d'où le tunnel ping pour contourner des restrictions.
- Cartographie de réseaux distants à la recherche de machines vulnérables : les pare-feux filtrent ICMP (pas d'Echo Reply, Unreachable venant toujours de la passerelle), parfois totalement.

23. DHCP

Source : `hta3-ipv4-protocols.pdf`, p. 8 à 10 ; `hta3p-ipv4-tools.pdf`, p. 14, 15 et 24 ; `ressources/demos/dhcp/`

> **À retenir**
>
> - Dynamic Host Configuration Protocol, RFC 2131. Le serveur attribue les adresses aux hôtes du réseau local.
> - Techniquement de la couche application, sur **UDP** : serveur port **67**, client port **68**.
> - Options TLV : masque, passerelle, DNS, serveur de temps, nom d'hôte, image de boot.
> - **État mou** : le bail (lease) doit être renouvelé, sinon il expire ; utiliser une adresse expirée pose problème si le serveur l'a donnée à un autre hôte.

Échange en quatre temps, **DORA** :

1. **Discover**, client vers tous : « qui est le serveur ? », broadcast liaison et IP.
2. **Offer**, serveur vers client : une adresse proposée (plusieurs serveurs possibles).
3. **Request**, en broadcast : « je prends celle-ci », tous les serveurs sont prévenus.
4. **Ack**, serveur vers client : confirmation.

Autres messages : decline, renew, release. (La p. 9 montre la RFC 2322, un « serveur DHCP » en pinces à linge.)

Outils et démo (`hta3p`)

Client `dhclient`, serveur `Kea`, tous deux de l'ISC. Démo Mininet : deux hôtes `na` et `nb` reliés par un lien (`ethAB` et `ethBA`) ; `na` reçoit 10.0.1.1/24 et lance `kea-dhcp4 -c kea.conf`, `nb` lance `dhclient -d ethBA`.

> **Lire `kea.conf` de la démo**
>
> Sous-réseau 10.0.1.0/24, pool 10.0.1.10 à 10.0.1.30 : $30 - 10 + 1 = 21$ adresses. Passerelle (`routers`) 10.0.1.1, l'adresse de `na`. Bail 20, renou-

vellement 10 : secondes selon la documentation de Kea (absent des supports). Extrait du fichier :

```
"Dhcp4": {
  "valid-lifetime": 20,
  "renew-timer": 10,
  "rebind-timer": 10,
  "interfaces-config": { "interfaces": [ "ethAB" ] },
  "subnet4": [ {
      "id": 1,
      "subnet": "10.0.1.0/24",
      "pools": [ { "pool": "10.0.1.10 - 10.0.1.30" } ],
      "option-data": [
        { "name": "routers", "data": "10.0.1.1" } ]
  } ]
}
```

C'est du JSON : un bon support pour s'entraîner à la conversion en YAML (concept 5).

24. ARP

Source : hta3-ipv4-protocols.pdf, p. 2 ; capture hta3p-ipv4-tools.pdf, p. 23

Ce que disent les slides. ARP est cité comme protocole « entre les couches », avec DHCP (hta3 p. 2). La capture `tcpdump` de hta3p p. 23 en montre un échange :

```
ARP, Request who-has 10.1.33.2 tell 10.1.33.1, length 28
ARP, Reply 10.1.33.2 is-at 24:5e:be:7b:be:c7, length 46
```

Aucune slide n'en décrit le fonctionnement. La fiche 2023 annonçait un TD sur ARP.

Fonctionnement standard (RFC 826, hors slides). Pour envoyer un paquet IP sur Ethernet, il faut l'adresse MAC du saut suivant : la destination si elle est dans mon sous-réseau, la passerelle sinon.

1. Consulter le cache ARP.
2. Sinon, diffuser en broadcast Ethernet une requête « qui a 10.1.33.2 ? dites-le à 10.1.33.1 ».
3. Le propriétaire répond en unicast avec sa MAC ; l'émetteur la met en cache (entrée à durée limitée).

ARP ne s'authentifie pas : une fausse réponse détourne le trafic (empoisonnement de cache). Ce point recoupe les modules ARP de CyberOps (BMEVIHIAV43).

Profondeur attendue à l'examen : **À VÉRIFIER**.

25. DNS

Source : `hta3-ipv4-protocols.pdf`, p. 11 à 18

À retenir

- Noms vers adresses IP, via `getaddrinfo`. Outils `host`, `nslookup`, `dig` ; serveurs BIND 9, Unbound, Dnsmasq.
- Ancêtre : `/etc/hosts`. RFC 882 et 883 (1983), puis RFC 1034 et 1035 (1987).
- Espace de noms en arbre. URL : `scheme://location/path/file` ; le DNS résout la partie location. URI : désignation plus générale.
- TLD : .com, .org, .edu, .gov, .net, ccTLD (.hu, .co.uk). FQDN : nom complet.
- Enregistrements : **A** (IPv4), **AAAA** (IPv6), **MX** (serveur de courrier), et d'autres.

Résolution (p. 14 et 15)

Base distribuée : chaque serveur tient un sous-arbre, par exemple tout ce qui finit par `.tmit.bme.hu`, et connaît certains serveurs au-dessus de lui ; cette topologie est indépendante du réseau. Le client connaît un résolveur local (souvent donné par DHCP) et demande les types voulus (A et AAAA). La requête monte puis redescend l'arbre jusqu'au serveur **autoritaire** ou jusqu'à une réponse en cache ; le client peut exiger une réponse autoritaire. Résolution inverse : d'une adresse IP vers un nom.

Organisation, transport, sécurité (p. 16 à 18)

- ICANN : espace d'adresses, numéros de protocole, TLD, serveurs racine. ISOC : .org. Registrars accrédités ; propriété vérifiable par WHOIS (RFC 3912).
- Transport : UDP/53 à l'origine, TCP/53 (messages longs, fiabilité, 1989), TLS (pas partout), HTTPS (sûr mais contourne le mécanisme de résolution d'origine).
- Faiblesses : ni chiffrement ni authentification, empoisonnement de cache, noms ressemblants (facebock.com, autre TLD, caractères grecs).

26. Outils de configuration réseau

Source : `hta3p-ipv4-tools.pdf`, p. 2 à 17

Windows : dialogue de paramètres réseau, `ipconfig` (état, client DHCP) ; côté serveur Active Directory (authentification, DHCP, DNS), IIS, RDS. Linux : noyau et outils GNU assemblés en distributions, installation par gestionnaire de paquets ; incontournable en réseau.

Commande (exemples des slides)	Rôle
`ifconfig eth0 10.0.3.14/24 up`	ajouter une adresse, activer ; `ifconfig eth0 down`
`route -n`	lister la table de relayage (numérique)
`route add 10.0.20.0/16 dev eth0`	ajouter une route
`netstat -pnaut`	connexions ouvertes : processus, numérique, tous états, UDP, TCP
`netstat -r, -i, -s`	routes, statistiques d'interfaces, de protocoles
`ip addr add 10.0.20.30/24 dev eth0`	iproute2 (Linux) : ajouter une adresse
`ip link set dev eth0 up`	activer une interface
`ip route add 10.20.30.0/24 dev eth0 via 10.0.20.10`	route via une passerelle

`ip` (iproute2) remplace `ifconfig` et `route` sur Linux ; ces deux-là existent sur tous les Unix, avec des options qui varient.

> **Piège**
>
> Dans `route add 10.0.20.0/16`, 10.0.20.0 n'est pas une adresse de réseau /16 : le réseau /16 qui la contient est 10.0.0.0/16. À l'examen, toujours vérifier qu'une route ou un réseau donné est aligné sur son préfixe.

Configuration permanente (p. 15 et 16)

```
# /etc/network/interfaces
auto eth0
auto eth1
iface eth0 inet dhcp
iface eth1 inet static
```

```
address 10.0.1.11/24
gateway 10.0.1.1
```

Le fichier `/etc/resolv.conf` contient la configuration DNS ; `dhclient` le gère automatiquement. Gestionnaires de réseau :

- NetworkManager, configuré avec `nmcli` ou une interface graphique ;
- systemd-networkd, règles dans `/etc/systemd/network`, avec son propre client DHCP ;
- netplan d'Ubuntu, règles dans `/etc/netplan/`, au-dessus des deux précédents.

Shell (p. 6 à 8) : interpréteur programmable (variables, `if`, `while`, `for`, fonctions), redirections > < et tubes `|`, tâches de fond. Commandes courantes : `man`, `cat`, `less`, `ls`, `find`, `sort`, `sed`, `ps`, `top`, `kill`, `sudo`, `lspci`…

27. Outils de diagnostic et de capture

Source : `hta3p-ipv4-tools.pdf`, p. 17 à 24

À retenir

- **ping** : Echo Request, la cible répond par Echo Reply. Un ping réussi ne garantit pas que le reste du trafic passe.
- **traceroute** : TTL croissant, on regarde qui renvoie Time Exceeded. Par défaut des paquets UDP vers un port aléatoire ; `-I` ICMP Echo Request ; `-T` TCP SYN. Le type de paquet compte, car les pare-feux ne les traitent pas pareil.
- **tcpdump** : enregistre et décode le trafic, sauvegarde dans un fichier (rejouable avec tcpreplay), mode promiscuous pour voir le trafic destiné à d'autres.
- **Wireshark** : équivalent graphique (ancien nom Ethereal, 2006) ; **tshark** en console, plus riche que tcpdump.
- **Mininet** : bac à sable de VM légères et liens virtuels, piloté en Python.

Lire la sortie de ping (p. 18)

```
64 bytes from ... (152.66.121.21): icmp_seq=1 ttl=60 time=1.43 ms
24 packets transmitted, 24 received, 0% packet loss
rtt min/avg/max/mdev = 1.062/1.282/1.443/0.097 ms
```

Par ligne : séquence, TTL restant, aller-retour. Bilan : aucune perte, RTT moyen 1,282 ms.

Lire traceroute (p. 19)

Chaque ligne numérotée est un saut, avec trois mesures de temps. Premier saut vers telex.hu : `_gateway (192.168.100.254)`, la passerelle par défaut. Vers isc.org en mode UDP, les sauts 9 à 30 affichent `* * *` : aucune réponse (filtrage) et la destination n'est jamais atteinte ; avec `-I` ou `-T`, elle répond au 10e saut.

Capture `tcpdump` (p. 23) : `ttl 64, id 15149, offset 0, flags [DF], proto ICMP (1), length 84`, les champs de l'en-tête IPv4.

28. Routage : rôle, tables, chemin optimal

Source : `hta4-routing.pdf`, p. 2 à 6 ; `hta4p-routing.pdf`, p. 2

> **À retenir**
>
> - **Relayage** (forwarding) : choisir le saut suivant d'un paquet à partir de ce que l'on sait de la topologie. **Routage** (routing) : découvrir la topologie.
> - Routage **statique** (configuré à la main) ou **dynamique** (découvert par un protocole de routage).
> - Deux bases séparées par rôle. **FIB** (Forwarding Information Base) : sert directement au choix du saut suivant, optimisée pour la vitesse de recherche. **RIB** (Routing Information Base) : sert à la découverte de la topologie, peut contenir des informations sans lien direct avec le relayage (méthode de découverte, délai d'expiration).

Routeur (p. 3) : dans le monde IP, tout nœud qui relaie du trafic ; en pratique il fait tourner des protocoles de routage pour remplir RIB et FIB. N'importe quel ordinateur à plusieurs interfaces peut l'être ; sous Linux, le relayage s'active par `/proc/sys/net/ipv4/ip_forward` (`hta4p`, p. 2). Capacité d'un routeur : nombre d'interfaces, bit/s (débit des interfaces, débit interne), paquets/s (dépend du traitement).

Chemin optimal (p. 5)

Optimal selon quel critère : moins de sauts, distance géographique, latence, bande passante, coût, fiabilité. Il faut **éviter les boucles**. Routage multichemin pour la fiabilité :

- redondance **1:1** : un chemin de secours, on bascule quand le principal tombe ;
- redondance **1+1** : le paquet part sur les deux chemins, perdre les deux copies est moins probable ;
- cela ne protège que si les chemins sont **disjoints**.

Sémantique du routage (p. 6)

Axe	Options
Communication	unicast, anycast, broadcast, multicast, geocast
Fonctionnement	saut par saut (chaque nœud décide) ; routage par la source (la source fixe la route, le réseau doit lui faire confiance)
Modèle	plat (un seul domaine, petits réseaux) ; hiérarchique (domaines, routage intra-domaine pour les nœuds d'un domaine, inter-domaine entre les passerelles de domaine)

29. Vecteur de distance et état des liens

Source : `hta4-routing.pdf`, p. 7 à 9 ; `hta4p-routing.pdf`, p. 3, 4 et 7

> **À retenir**
>
> DV : « je dis à mes voisins qui je connais ». LS : « je dis à tout le monde qui sont mes voisins » (`hta4`, p. 9).

	Distance vector (DV)	Link state (LS)
Algorithme	Bellman-Ford distribué	Dijkstra
Ce qui est envoyé	la liste des destinations connues avec leur distance	l'état de mes liens : qui je suis, qui sont mes voisins (et le poids)
À qui	aux voisins, périodiquement	à tous les nœuds connus, périodiquement
Calcul	je choisis comme saut suivant le voisin le plus proche de la destination	quand j'ai tout l'état des liens, recherche pondérée (Dijkstra) sur la topologie
Convergence	lente : à chaque ronde on apprend des destinations plus lointaines ; diamètre de 10 sauts, 10 rondes	détecte et reconfigure plus vite qu'un DV après une panne de lien
Exemples	RIP, IGRP	OSPF, IS-IS

Étapes (`hta4p`, p. 4 et 7)

DV. (1) Découvrir les voisins, joignables en 1 saut. (2) Dire aux voisins quels nœuds on joint ; au début, nos voisins directs, qui apprennent ainsi ce qu'ils joignent en 2 sauts par nous. Répéter (2) : si A annonce C à x sauts et B à $y < x$, B devient le saut suivant vers C, à $y + 1$. Les liens pondérés (capacité, latence) sont possibles mais rares en DV.

LS. (1) Découvrir les voisins, construire l'état des liens (OSPF : protocole Hello). (2) Échanger l'état des liens avec tous les nœuds connus (OSPF : LSA). (3) On connaît alors les voisins de nos voisins : échanger avec ces nouveaux nœuds, jusqu'à les avoir tous. (4) Lancer Dijkstra sur la topologie obtenue.

Pseudo-code des slides (hta4p, p. 3)

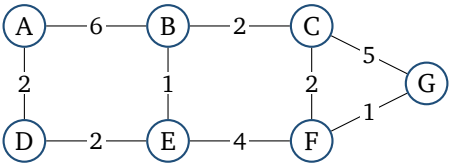
```
Dijkstra, file de priorité Q
forall v in V:
  if v != source:
    v.dist = inf, v.prev = undef
    Q.add(v, v.dist)
source.dist = 0, Q.add(source, 0)
while Q is not empty:
  u = Q.get_min_prio()
  forall neighbor v of u:
    d = u.dist + dist(u, v)
    if d < v.dist:
      v.dist = d, v.prev = u
      Q.change(v, d)
```

```
Bellman-Ford
forall v in V:
  if v != source:
    v.dist = inf, v.prev = undef
source.dist = 0
repeat |V|-1 times:
  forall edge (u, v), weight w:
    if u.dist + w < v.dist:
      v.dist = u.dist + w
      v.prev = u
```

Complexités : Dijkstra $O(|E| + |V| \log |V|)$, Bellman-Ford $O(|V|^2)$ selon la slide ; en LS on prend le plus rapide. Bellman-Ford se distribue, d'où son usage en DV.

30. Méthode : Dijkstra pas à pas

Source : `hta4p-routing.pdf`, p. 8 à 14 (exemple des slides, recalculé par `calc_net_s46.py`)



Base d'état des liens de la slide p. 8 : A : B/6, D/2 ; B : A/6, C/2, E/1 ; C : B/2, F/2, G/5 ; D : A/2, E/2 ; E : B/1, D/2, F/4 ; F : C/2, E/4, G/1 ; G : C/5, F/1.

À retenir

1. Source à 0, tous les autres à ∞.
2. Retirer de la file le nœud de plus petite distance : sa distance est définitive.
3. Pour chacun de ses voisins non définitifs : si (distance du nœud retiré + poids du lien) est plus petite que la distance connue, la remplacer et noter le prédécesseur.
4. Recommencer jusqu'à file vide. Le premier saut se lit en remontant les prédécesseurs.

Lecture du tableau : chaque ligne donne l'état après traitement du nœud retiré. Une case « d,p » signifie distance d, prédécesseur p ; souligné : valeur améliorée à cette étape ; **gras** : distance définitive. En cas d'égalité, la slide retire B avant F (p. 10) et E avant G (p. 12) : l'ordre alphabétique reproduit ce choix et ne change pas les distances.

Étape	Retiré	A	B	D	E	F	G
0	C (0)	∞	2,C	∞	∞	2,C	5,C
1	B (2)	8,B	**2**	∞	3,B	2,C	5,C
2	F (2)	8,B		∞	3,B	**2**	3,F
3	E (3)	8,B		5,E	**3**		3,F
4	G (3)	8,B		5,E			**3**
5	D (5)	7,D		**5**			
6	A (7)	**7**					

Les étapes collent aux slides : étape 1, C-B-A = 8 et C-B-E = 3 (p. 10) ; étape 2, C-F-G = 3 remplace 5 (p. 11) ; étape 3, C-B-E-D = 5, et C-B-E-F = 7 est rejeté car F est déjà à 2 (p. 12) ; étape 5, C-B-E-D-A = 7 remplace 8 (p. 14).

Destination	Distance	Premier saut	Chemin
A	7	B	C → B → E → D → A
B	2	B	C → B
D	5	B	C → B → E → D
E	3	B	C → B → E
F	2	F	C → F
G	3	F	C → F → G

Piège

- Le chemin le plus court vers A ne passe pas par le lien direct B-A (poids 6) : $2 + 1 + 2 + 2 = 7 < 8$.
- Le routeur ne garde dans sa FIB que le **premier saut** (B ou F ici), pas le chemin.
- Un nœud retiré n'est plus jamais mis à jour : ne pas « améliorer » B à l'étape 3.

31. Méthode : vecteur de distance et count to infinity

Source : `hta4p-routing.pdf`, p. 4 à 6 ; tableaux recalculés par `calc_net_s46.py`

Exemple de la chaîne A-B-C-D-E-F (p. 5)

Au départ personne ne connaît rien et chacun diffuse une table vide ; dans cette topologie, une diffusion atteint les voisins. Après la 1re diffusion, chacun connaît ses voisins directs (C : B et D à 1 saut). À la 2e, C reçoit de D `[[C,1],[E,1]]` et apprend E à 2 sauts par D. Ensuite C annonce `[[A,2],[B,1],[D,1],[E,2]]`. L'annonce contient destination et distance, **pas le saut suivant**, que C garde pour lui. Table de C :

Après la diffusion	A	B	D	E	F
1		1 via B	1 via D		
2	2 via B	1 via B	1 via D	2 via D	
3	2 via B	1 via B	1 via D	2 via D	3 via D
4	2 via B	1 via B	1 via D	2 via D	3 via D
5	2 via B	1 via B	1 via D	2 via D	3 via D

Après la 5e diffusion, A apprend F (et F apprend A) : c'est fini, car A et F sont à 5 sauts.

Règle de calcul (rappel général, équation de Bellman-Ford, qui est ce que décrit la p. 4) : $d_X(Y) = \min_{v \text{ voisin de } X} \big(c(X,v) + d_v(Y)\big)$, où $c(X,v)$ est le coût du lien vers le voisin et $d_v(Y)$ la distance annoncée par ce voisin. En RIP, $c = 1$.

Count to infinity (p. 6)

Même chaîne, topologie entièrement découverte : C joint A en 2 sauts par B, D joint A en 3 sauts par C. B tombe ; C le détecte et marque inaccessibles A et B. C reçoit alors l'annonce de D « A à 3 » : il ne sait pas qu'il est lui-même le saut suivant de D, et passe à 4 via D. Il annonce 4 à D ; C a toujours été le saut suivant de D vers A, donc D croit que quelque chose a changé au-delà de C et passe à 5. Et ainsi de suite :

Message	Annonce	C vers A	D vers A
panne de B		inaccessible	3 via C
1	D annonce A à 3	4 via D	3 via C
2	C annonce A à 4	4 via D	5 via C
3	D annonce A à 5	6 via D	5 via C
4	C annonce A à 6	6 via D	7 via C
5	D annonce A à 7	8 via D	7 via C
6	C annonce A à 8	8 via D	9 via C
7	D annonce A à 9	10 via D	9 via C
8	C annonce A à 10	10 via D	11 via C
9	D annonce A à 11	12 via D	11 via C
10	C annonce A à 12	12 via D	13 via C
11	D annonce A à 13	14 via D	13 via C
12	C annonce A à 14	14 via D	15 via C
13	D annonce A à 15	16 (infini RIP)	15 via C
14	C annonce A à 16	16 (infini RIP)	16 (infini RIP)

La boucle ne s'arrête que parce que RIP plafonne la métrique : limite de 15 sauts (`hta4`, p. 10) ; la valeur 16 signifie « infini », donc inaccessible (rappel général, RFC 1058). Il faut ici 14 annonces ; avec une annonce toutes les 30 s (p. 10), cela se compte en minutes.

À retenir

Solutions de la slide, sans ajouter d'information aux annonces :

- **Split horizon** : ne pas annoncer une route au voisin qui est son saut suivant (D ne dit pas à C qu'il joint A, puisque C est son saut suivant).
- **Poisoned reverse** : annoncer à son saut suivant la route avec un coût infini ; il ne nous choisira pas comme saut suivant.

Avec l'un ou l'autre, C ne reçoit plus « A à 3 » de D : A reste inaccessible chez C, D l'apprend à l'annonce suivante de C, et la panne est propagée sans boucle.

Piège

Rappel général : split horizon et poisoned reverse suppriment les boucles entre deux voisins, pas toutes les boucles de trois routeurs ou

> plus. La slide ne traite que le cas de deux nœuds.

32. RIP, OSPF, IS-IS et la démo BIRD

Source : `hta4-routing.pdf`, p. 10 à 12 ; `hta4p-routing.pdf`, p. 15 à 17 ; démo `demos/routing`

	RIP	OSPF	IS-IS
Type	DV	LS	LS
Norme	RFC 1058 (1988) ; RIPv2 RFC 1723 (1994) ; RIPng RFC 2080 (1997)	RFC 1131 (1989) ; OSPFv2 RFC 2328 (1998) ; OSPFv3 IPv6 RFC 5340 (2008)	ISO 10589:2002 ; RFC 1142 révoquée par RFC 7142
Métrique	nombre de sauts, limite 15	coût du chemin, en général bande passante relative à une référence	
Détails	mise à jour toutes les 30 s ; v2 : CIDR, convergence plus rapide, authentification ; RIPng : IPv6, authentification par IPsec	Hello pour les voisins ; LSA (LS Request, LS Update, LS Acknowledgement) ; aires	pas lié à IP (toute couche 3, aussi Shortest Path Bridging IEEE 802.1aq) ; pas d'aire dorsale centrale ; messages plus faciles à étendre
Usage		réseaux d'entreprise	réseaux d'opérateurs

OSPF en détail (p. 11)

- Le réseau est découpé en **aires** ; les LSA restent dans l'aire. Aire dorsale (backbone, 0.0.0.0) qui relie toutes les autres ; aire normale ; aire de transit, sans station terminale, parmi lesquelles on choisit en général la dorsale.
- L'identifiant d'aire fait 32 bits et s'écrit a.b.c.d, mais **ce n'est pas une adresse IP**.
- Sur un lien qui n'est pas point à point (Ethernet), plusieurs routeurs se partagent le segment : on élit un **Designated Router** qui représente le segment, et un Backup Designated Router.

Démo (`hta4p`, p. 15 à 17)

Quatre hôtes Mininet en carré, tous les réseaux en /24 : na (10.0.1.1, 10.0.4.1), nb (10.0.1.2, 10.0.2.2), nc (10.0.2.3, 10.0.3.3), nd (10.0.3.4,

10.0.4.4). Sans routage, `na ping nb` marche, `na ping nd` (10.0.3.4) et `na ping nc` échouent. Après démarrage de BIRD (démon qui implémente RIP, OSPF, BGP, remplit la FIB du noyau, vérifiable par `ip route`, supervisé par `birdc`), tout le monde se joint.

Fichiers de la démo : `bird_rip.conf` (`version 2`, `update time 10`, `timeout time 60`, `split horizon on`) et `bird_ospf.conf` (une seule aire `0.0.0.0`, `cost 5` par interface, `type broadcast`, `hello 5`, `dead 20`).

Implémentations (p. 16) : Cisco IOS ; Juniper JunOS (base BSD) ; logiciels libres Quagga (successeur de Zebra), FRRouting (fork de Quagga), XORP, BIRD.

33. Topologie d'Internet, AS et BGP

Source : `hta4-routing.pdf`, p. 13 à 15 et 17

> **À retenir**
>
> - **AS** (Autonomous System) : réseau local d'un fournisseur d'accès ou d'une entreprise ; Internet est un ensemble d'AS reliés. Avantages : petits morceaux plus simples à exploiter, meilleur passage à l'échelle. Inconvénient : difficile à exploiter à l'échelle mondiale.
> - **IGP** (Interior Gateway Protocol), dans un AS : RIP, OSPF, IS-IS…**EGP** (Exterior Gateway Protocol), entre AS : BGP.

Relations entre AS (p. 14)

Topologie hiérarchique. **Peering** : liaison entre AS de même niveau ; relation par paire, **non transitive**. **Transit** : un AS de niveau supérieur relie des AS de niveau inférieur. Ces liaisons sont établies à la main, par contrat entre entreprises (il y a de l'argent en jeu). Exemple donné : BIX (Budapest Internet Exchange).

BGP (p. 15)

- Border Gateway Protocol, le protocole de routage inter-AS. RFC 1105 (1989) ; BGPv4 RFC 1654 (1994), RFC 4271 (2006).
- **Path vector** : proche du DV, mais on échange des chemins entiers, pas seulement avec les voisins.
- **Routage par politique** : découverte automatique et directives ajoutées à la main. Le coût d'un chemin n'est pas seulement bande passante ou latence, mais de l'argent (on paie au volume) ; confiance limitée entre AS (contourner un pays soupçonné d'espionnage).
- **iBGP** : BGP comme protocole intérieur ; peu efficace car il faut un maillage complet. Remède : **Route Reflector**, un nœud représente une grappe, maillage complet seulement entre RR.

Piège

La p. 17 (« Sensor Networks » : DSDV, OLSR, DSR, AODV, B.A.T.M.A.N.) est marquée « Not exam material ».

34. Couche transport et ports

Source : `hta5-l4.pdf`, p. 2 à 6

Dans le modèle IP (p. 2) : la session OSI (connexion, déconnexion, keepalive) passe en partie dans TCP, mais le keepalive est à la charge de l'application ; le transport OSI (échange fiable de segments, multiplexage) devient la couche transport TCP/IP.

Couche	Identifiant	Taille
Liaison	adresse MAC : interface, localement	48 bits
Réseau	adresse IP : interface, globalement	32 ou 128 bits
Session (slide)	port : connexion réseau d'une application	16 bits

À retenir

- Ports nécessaires parce que plusieurs applications ont des connexions en même temps, et qu'une application peut en avoir plusieurs.
- A initie, B répond : B ouvre un **port d'écoute** que A connaît. Ports **bien connus** 0 à 1023, attribués par l'IANA aux serveurs courants (sous Unix, il faut être superutilisateur pour en ouvrir un) ; un autre port peut servir de port d'écoute.
- A prend n'importe quel port source, B répond vers ce port. **Port éphémère** : le système choisit un port libre au hasard (exemple : HTTP). Certains protocoles imposent les deux ports (DHCP : serveur 67, client 68).
- Une conversation est identifiée par le **5-uplet** {IP 1, IP 2, protocole, port 1, port 2}, protocole = UDP, TCP…

Contrôle de flux et de congestion (p. 5)

Flux : le récepteur n'a pas une capacité de traitement infinie ; surchargé, il demande à l'émetteur de ralentir. **Congestion** : le réseau n'a pas une capacité infinie, d'autres l'utilisent, la congestion peut survenir partout ; il faut ralentir pour éviter les pertes. Détection : latence en hausse, paquets jetés (voir RED).

Datagrammes et flux (p. 6)

Les données voyagent en paquets ; au transport, des segments (en pratique, presque toujours 1 segment = 1 paquet). **Datagramme** : chaque message est un segment, remis tout de suite à l'application. **Flux** (stream) : suite continue d'octets ; quand la remettre (tout de suite, sur marqueur de fin de message, tampon plein) ? Un protocole de transport n'implémente en général que l'un des deux.

35. UDP

Source : `hta5-l4.pdf`, p. 7

User Datagram Protocol, RFC 768, numéro de protocole IP **17**. Sans connexion, orienté datagramme, sans fiabilité, sans contrôle de flux ni de congestion. Ce qu'il ajoute à IP : **les ports**.

Bit	0 à 15	16 à 31
0	Source Port	Destination Port
32	Length	Checksum

- En-tête de 8 octets (deux mots de 32 bits).
- **Length** : « inutile » selon la slide, IP la contient déjà. Rappel général : elle compte en-tête et données, donc au moins 8.
- **Checksum** : couvre aussi une partie de l'en-tête IP (adresses, longueur, protocole) : violation de la séparation des couches. Checksum = 0 est toujours valide en IPv4 (pas de contrôle), **plus en IPv6**.

36. TCP : connexion

Source : `hta5-l4.pdf`, p. 8 à 10

Transmission Control Protocol : RFC 675 à l'origine, RFC 793 pour IPv4, spécification actuelle RFC 9293 (remplace toutes les RFC d'extension), numéro de protocole IP **6**. Orienté connexion, bidirectionnel, orienté flux, fiable (acquittement, retransmission sur numéros de séquence), contrôle de flux et de congestion, ni broadcast ni multicast.

Client	sens	Serveur
SYN (Sc)	→	
	←	SYN, ACK (Ss, Sc+1)
ACK (Sc+1, Ss+1)	→	
données	↔	données
FIN	→	
	←	FIN, ACK
ACK	→	

À retenir

- Poignée de main en 3 temps à l'ouverture et à la fermeture, avec les drapeaux SYN, ACK, FIN ; elle fixe aussi les numéros de séquence initiaux, **choisis au hasard**.
- Champ Sequence : avec SYN = 1, valeur initiale ; avec SYN = 0, séquence du segment envoyé. Champ Acknowledgement : avec ACK = 1, prochaine séquence attendue par le récepteur ; avec ACK = 0, rien.
- Chacune des deux parties peut lancer la déconnexion.

Rappel général (RFC 9293) : les numéros comptent des **octets** ; SYN et FIN consomment chacun un numéro, d'où le +1. Après la poignée de main, le premier octet de données du client porte Sc+1 ; s'il envoie 100 octets, l'acquittement vaut Sc+101. La fermeture se fait souvent en quatre segments (FIN, ACK, puis FIN, ACK dans l'autre sens) ; la slide la montre en trois, le FIN et l'ACK du serveur étant regroupés.

Sécurité (p. 10)

Le serveur répond SYN+ACK et doit retenir les numéros de séquence pour vérifier l'ACK du client, dans une table de taille limitée. **SYN flood** : envoyer énormément de SYN sans intention de se connecter, pour épuiser la table. Défense, **SYN cookies** : ne pas retenir le numéro choisi mais le générer par une fonction cryptographique, et vérifier l'ACK avec la même fonction. Autres attaques : détournement de connexion, reset.

37. TCP : acquittement, fenêtre, en-tête

Source : `hta5-l4.pdf`, p. 11 à 16 ; `hta5p-tcp.pdf`, p. 2

Acquittement et fenêtre (p. 11)

- Tout segment reçu doit être acquitté ; tout segment avec ACK = 1 peut le faire. En communication à sens unique, le récepteur envoie des segments vides rien que pour acquitter.
- Un ACK signifie : tout jusqu'à Acknowledgement −1 est arrivé ; Acknowledgement est le prochain attendu.
- L'émetteur n'attend pas l'ACK pour envoyer la suite : **fenêtre d'émission** glissante, seuls les segments dans la fenêtre partent ; elle avance quand son premier segment est acquitté. Sa taille (segments « en vol ») règle indirectement le débit ; contrôle de flux et de congestion jouent dessus.
- TCP d'origine : chaque segment acquitté individuellement. **Selective ACK** : acquitter une série de segments d'un coup. Sans acquittement avant l'expiration du délai : retransmission (`hta5p`, p. 2).

En-tête (p. 13 à 16)

Bit	0 à 3	4 à 6	7 à 15	16 à 31
0	Source Port			Destination Port
32	Sequence Number			
64	Acknowledgement Number			
96	Data Offset	Reserved	Flags (9 bits)	Window Size
128	Checksum			Urgent Pointer
160	Options (si Data Offset > 5)			

Champ	Rôle selon les slides
Ports	identifient la connexion
Sequence, Ack	voir ci-dessus
Data Offset	indique la longueur des options. Rappel général : longueur de l'en-tête en mots de 32 bits, 5 sans option (20 octets), 15 au plus (60 octets)
Window Size	fenêtre de réception annoncée par le récepteur, pour que l'émetteur sache combien envoyer (contrôle de flux)
Checksum	même principe qu'UDP
Urgent Pointer	fin de la partie urgente (avec URG)

Drapeaux (neuf bits, six importants, p. 14) : **SYN** synchroniser les numéros de séquence (ouverture seulement) ; **FIN** terminer ; **ACK** acquitter ; **RST** « reset the window to minimum size (on error) » selon la slide (rappel général : dans la RFC 9293, RST interrompt la connexion) ; **PSH** remettre à l'application les données déjà reçues (fin de message) ; **URG** segment important jusqu'à l'Urgent Pointer.

Rappel général, pour décoder un en-tête en hexadécimal : octet 12 = Data Offset (4 bits de poids fort) ; octet 13, du poids fort au poids faible : CWR, ECE, URG, ACK, PSH, RST, SYN, FIN. Exemple : `0x12` = ACK + SYN, `0x11` = ACK + FIN, `0x02` = SYN.

Options (TLV, p. 16) :

- **Window scale** : la fenêtre sur 16 bits plafonne à 64 ko ; facteur de décalage de la fenêtre, au plus 14 bits, donc 16 + 14 = 30 bits : fenêtre maximale de 1 Go.
- **Selective acknowledgement** : acquitter une plage de séquence d'un coup.
- **Timestamp** : aide à rétablir l'ordre quand grandes fenêtres et retour à zéro des numéros de séquence le rendent ambigu (PAWS, Protection Against Wrapped Sequence).

> **Piège**
>
> La p. 17 (« Jokes ») est marquée « Not exam material ».

38. NAT

Source : `hta5-l4.pdf`, p. 18 et 19

Les plages privées (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) ne sont pas routables sur Internet. **Network Address Translation** : un nœud du réseau privé ouvre une connexion vers un serveur d'Internet ; la passerelle remplace

- l'adresse IP source par sa propre adresse externe, publique ;
- le port source par un nouveau port éphémère,

et retient l'association port éphémère → {IP source d'origine, port source d'origine}. Le serveur répond à l'IP et au port de la passerelle, qui réécrit selon l'association retenue.

Adresses de la slide, ports choisis pour l'exemple

Client 192.168.1.31, passerelle 192.168.1.1 (interne) et 94.44.112.4 (externe), serveur 152.66.244.17. Le client ouvre une connexion depuis son port 50000 vers le port 443 ; la passerelle choisit le port 40001.

Tronçon	Source	Destination
client → passerelle	192.168.1.31:50000	152.66.244.17:443
passerelle → serveur	**94.44.112.4:40001**	152.66.244.17:443
serveur → passerelle	152.66.244.17:443	94.44.112.4:40001
passerelle → client	152.66.244.17:443	**192.168.1.31:50000**

Table de la passerelle : 40001 ↔ 192.168.1.31:50000.

À retenir

- Ne marche que pour les connexions **sortantes** : c'est le S-NAT (Source NAT). Pour joindre un serveur du réseau privé, on configure un **D-NAT**.
- Utile : plus d'ordinateurs que d'adresses IP ; réseau caché aux requêtes extérieures.
- Problèmes : nombre limité de ports éphémères sur la passerelle ; applications qui veulent une connexion directe entre adresses publiques (pair à pair, VoIP H.323) ; applications qui signent cryptographique-

ment les adresses des extrémités contre le MitM.

- **NAT traversal** : demander à la passerelle d'ouvrir un port pour les connexions entrantes (UPnP Internet Gateway Protocol).

39. Autres protocoles de transport

Source : `hta5-l4.pdf`, p. 20 à 23

TCP et UDP sont les deux extrémités d'un spectre ; il existe des intermédiaires.

Protocole	Référence	Caractéristiques
TCP	RFC 9293, proto 6	avec connexion, flux, fiable, ordonné
UDP	RFC 768, proto 17	sans connexion, datagramme, non fiable, non ordonné
DCCP	RFC 4340, proto 33	« UDP amélioré » : ouverture de connexion fiable ; acquittement mais pas de retransmission ; notification de congestion mais pas de contrôle automatique
SCTP	RFC 9260, proto 132	remplaçant de TCP : orienté datagramme, livraison ordonnée ou non, multihoming et multichemin natifs ; conçu pour SS7 sur IP, surtout utilisé en télécom
DIY	sur UDP	son propre contrôle de congestion au-dessus d'UDP : QUIC pour HTTP/3, μTP pour BitTorrent

Pourquoi au-dessus d'UDP ? Ouvrir des sockets autres que TCP ou UDP demande des privilèges particuliers (p. 23).

40. Contrôle de flux et de congestion TCP

Source : `hta5p-tcp.pdf`, p. 2 à 7

Contrôle de flux (p. 3)

- Récepteur surchargé : il annonce une fenêtre plus petite, `adv_wnd`.
- Le champ Window Size vaut toujours le nombre d'**octets libres** du tampon de réception : ce que l'émetteur peut envoyer au plus avant d'attendre un acquittement.
- `adv_wnd` = 0 (tampon plein) : l'émetteur s'arrête un moment, puis envoie un petit segment pour obtenir une taille de fenêtre dans l'acquittement.
- **Silly Window Syndrome** : émetteur rapide qui remplit le tampon, le récepteur annonce une petite fenêtre, aussitôt remplie, puis une plus petite… On finit avec une petite fenêtre sans bonne raison. Autre cause : l'application émettrice fait de petites écritures que TCP envoie aussitôt. Remèdes : côté émetteur, regrouper les petites écritures (**algorithme de Nagle**) ; côté récepteur, n'annoncer une fenêtre non nulle que lorsque le tampon est à moitié vide.

Congestion (p. 4 à 6)

Goulot : la plus petite capacité disponible sur le chemin, pas forcément le premier lien, inconnue de l'émetteur, d'où le besoin de retour du récepteur. Causes : capacité limitée, flux concurrents qui veulent tous le débit maximal ; charge non uniforme (cycle d'activité quotidien, nouveaux goulots).

Effondrement : la congestion provoque des pertes, donc des retransmissions, qui augmentent la charge et les pertes, et la latence. **Goodput** : part utile du trafic (arrivée à destination, pas un doublon) ; idéalement goodput = charge. Au-delà du **genou** (knee), le goodput n'est plus proportionnel à la charge et la latence explose ; au-delà de la **falaise** (cliff), effondrement. But : rester à gauche du genou, tout près.

Pour l'éviter : ne rien faire (bloqué en faible goodput) ; réserver de la bande passante par flux (évitement parfait, mais problème d'empaquetage NP-complet, ne passe pas à l'échelle, latence d'établissement) ; **ajuster dynamiquement le débit** (accélérer sans mauvais retour, ralentir à la congestion : imparfait mais marche en pratique).

Le contrôle de congestion TCP (p. 7)

À retenir

- Côté émetteur, à partir des numéros d'acquittement. Unité : le **MSS** (Maximum Segment Size), en général la MTU du chemin, parfois moins.
- Fenêtres : `adv_wnd` (contrôle de flux), `cwnd` (contrôle de congestion), **fenêtre effective** = min(`adv_wnd`, `cwnd`), celle qui règle l'émission.
- Détection de la congestion par la perte : expiration du délai avant l'ACK (timeout), ou **ACK dupliqué** (on envoie de nouveaux segments mais le récepteur attend toujours le même).
- Deux phases : **slow start** (pas lent : petite fenêtre au départ, croissance exponentielle) puis **congestion avoidance** (rester près du genou estimé, AIMD).

41. Méthode : slow start et congestion avoidance

Source : `hta5p-tcp.pdf`, p. 8 à 10 ; tableaux recalculés par `calc_net_s46.py`

À retenir

- **Slow start** (p. 8) : `ssthresh` est l'estimation du genou. Départ avec `cwnd` = 1 et `ssthresh` = `adv_wnd`. Après chaque ACK réussi, `cwnd` ×2. Quand `cwnd` atteint `ssthresh` : passage en congestion avoidance. Sur timeout : recommencer à 1 avec `ssthresh`/2.
- **Congestion avoidance** (p. 9) : après chaque ACK réussi, `cwnd` +1, et `ssthresh` = `cwnd`. Sur timeout : `ssthresh`/2, retour en slow start.
- **AIMD** : croissance additive prudente, décroissance multiplicative agressive. Après le reset, `ssthresh` n'est plus `adv_wnd` : c'est une nouvelle estimation du genou.

Conventions de calcul. Comme sur les schémas des p. 8 et 9, une ligne par aller-retour (« ronde ») et `cwnd` en segments : la ronde envoie `cwnd` segments et l'ACK de la ronde fait évoluer `cwnd`. En congestion avoidance, `ssthresh` suit `cwnd`, donc « `ssthresh`/2 » au timeout revient à la moitié de la fenêtre au moment de la perte. Si le doublement dépasserait `ssthresh`, on s'arrête à `ssthresh` (convention d'exercice ; la slide dit seulement « quand cwnd atteint ssthresh »).

Numéros de segments des slides p. 8 et 9 (ssthresh = 8)

Ronde	cwnd	ssthresh	Phase	Segments	Événement
1	1	8	SS	1	
2	2	8	SS	2 à 3	
3	4	8	SS	4 à 7	
4	8	8	CA	8 à 15	
5	9	9	CA	16 à 24	
6	10	10	CA	25 à 34	

ssthresh = 16, timeout pendant la ronde 9 (valeurs choisies)

Ronde	cwnd	ssthresh	Phase	Événement
1	1	16	SS	
2	2	16	SS	
3	4	16	SS	
4	8	16	SS	
5	16	16	CA	
6	17	17	CA	
7	18	18	CA	
8	19	19	CA	
9	20	20	CA	timeout
10	1	10	SS	
11	2	10	SS	
12	4	10	SS	
13	8	10	SS	
14	10	10	CA	

Au timeout, `cwnd` = 20 et `ssthresh` = 20 (il suit `cwnd`) : nouveau `ssthresh` = 10, `cwnd` = 1. En ronde 14, le doublement donnerait 16 : plafonné à 10, on repasse en congestion avoidance.

Rappel général (RFC 5681) : dans la norme, slow start ajoute 1 MSS par ACK reçu, ce qui double la fenêtre à chaque aller-retour ; congestion avoidance ajoute environ 1 MSS par aller-retour ; au timeout, `ssthresh` = max(données en vol/2, 2 MSS) et `cwnd` = 1 MSS. Les conventions ci-dessus en sont la version « une ligne par ronde » des slides.

Schéma de la p. 10 (Tahoe) : dents de scie, chaque timeout ramène `cwnd` à 1 et une nouvelle montée exponentielle jusqu'au nouveau `ssthresh`, puis linéaire. Les connexions courtes restent en slow start : HTTP en souffre (p. 8).

42. TCP Reno, variantes et CUBIC

Source : `hta5p-tcp.pdf`, p. 11 à 15, 18 à 21 ; complément : TD 2019 (`HTA_GYAK_ALL_2019`), p. 35 à 39

Le contrôle vu jusqu'ici est l'original, nom de code **Tahoe** (BSD 4.3, 1988). Première amélioration, **Reno** (1990) : fast retransmit et fast recovery.

Fast retransmit (p. 13)

On n'attend pas le timeout : **3 ACK dupliqués** signifient que le récepteur attend toujours le même segment, qui n'est pas arrivé ; on le retransmet aussitôt. Réaction bien plus rapide que Tahoe. Figure de la slide, segment 21 perdu (ACK = prochain segment attendu) :

Segment émis	Récepteur	Réponse
20	reçu	ACK 21
21	perdu	
22	reçu	ACK 21 (dupliqué 1)
23	reçu	ACK 21 (dupliqué 2)
24	reçu	ACK 21 (dupliqué 3)
25	reçu	ACK 21 (dupliqué 4)
21	reçu	ACK 26
26	reçu	ACK 27

L'émetteur retransmet 21 au 3e doublon (reçu en réponse à 24) ; sur la figure, la retransmission part après 25, déjà en route. Quand 21 arrive, le récepteur a tout jusqu'à 25 et acquitte directement 26.

Fast recovery (p. 14)

> **À retenir**
>
> Perte détectée par 3 ACK dupliqués en congestion avoidance : retransmettre (fast retransmit), **ne pas** revenir en slow start, **ne pas** remettre `cwnd` à 1 ; `cwnd`/2 et rester en congestion avoidance (+1 par ronde), `ssthresh` suivant toujours `cwnd`. Sur **timeout**, retour en slow start : la congestion semble grave.

Le TD 2019 (p. 36) formule la même règle avec « cwnd = ssthresh/2 » ; avec la convention des slides 2026 où `ssthresh` suit `cwnd`, cela revient au même. Schéma p. 15 : la quantité de données transmises est l'aire sous la courbe, bien plus grande qu'avec Tahoe, car la phase de congestion avoidance oscille près du genou sans coupe excessive.

Variantes (p. 17 à 19)

Variante	Idée
Tahoe (1988)	contrôle d'origine : slow start, AIMD
Reno (1990)	+ fast retransmit, fast recovery
New Reno (1999)	fast recovery améliorée : retransmet au premier ACK dupliqué et garde `cwnd` au lieu d'attendre 3 doublons et de diviser ; des ACK hors d'ordre peuvent déclencher trop de retransmissions ; longtemps très répandu
Vegas (1994)	détecte la congestion par la hausse du délai des ACK, bien plus tôt que Reno, et perd du débit face à lui
Compound	une fenêtre sur les pertes et une sur le délai (Windows Vista)
BIC	meilleur que Reno sur LFN ; croissance exponentielle sans perte ; recherche binaire de la bonne `cwnd` entre une taille avec perte et une sans
CUBIC	BIC amélioré : `cwnd` fonction cubique du temps depuis la dernière congestion ; se comporte bien avec RTT courts et longs car presque indépendant du RTT. A remplacé BIC dans Linux 2.6.19 (2006), New Reno dans OS X 10.10 (2014), Compound dans Windows 10 (2017)

Ce qu'il faut savoir (p. 20 et 21)

Programmeur : l'ouverture TCP est lente (poignée de main TCP, puis TLS), donc réutiliser les connexions (HTTP/1.1) ; TCP est un flux, les lectures ne correspondent pas aux écritures, il faut lire jusqu'à la fin du message (`</html>`) ; PSH et URG peu utilisés ; le débit dépend des lectures du récepteur et de l'état du réseau ; options de socket ; après fermeture, le port reste occupé un moment (sauf `SO_REUSEPORT`). **Opérateur** : trafic en rafales (files très variables) ; TCP ralentit sur perte (RED jette tôt) ; TCP remplit toute la bande passante (dimensionnement difficile) ; les variantes ne cohabitent pas bien (Compound a évincé les variantes Reno à la sortie de Vista).

43. Produit bande passante délai, liens à pertes

Source : `hta5p-tcp.pdf`, p. 16 et 17 ; `hta5-l4.pdf`, p. 16 ; complément : TD 2019, p. 40 et 55

Long Fat Network (LFN, p. 16) : grande capacité et grand RTT (câble sous-marin, Ethernet 100 Gbit/s, cœur d'Internet). TCP s'y comporte mal : il faut attendre l'acquittement avant d'envoyer de nouveaux segments, et la quantité non acquittée est plafonnée par la fenêtre. Solution : **window scaling** (option en fin d'en-tête, décalage d'au plus 14 bits, fenêtre de 1 Go, 30 bits).

À retenir

Méthode générale (le TD 2019, p. 40, nomme le « bandwidth delay product ») :

- BDP = débit × RTT : la quantité de données « en vol » qui remplit le tuyau. Fenêtre optimale ≥ BDP.
- Avec une fenêtre W, débit maximal $\approx W/\text{RTT}$, quel que soit le lien.
- Sans option, $W \leq 65\,535$ octets ; avec un décalage s, $W \leq 65\,535 \times 2^s$.

Lien de 100 Mbit/s, RTT 80 ms (valeurs choisies)

BDP $= 100 \times 10^6 \times 0{,}08 = 8 \times 10^6$ bits $= 1\,000\,000$ octets, soit 685 segments de 1460 octets. Sans window scale : $65\,535 \times 8/0{,}08 \approx 6{,}55$ Mbit/s, 6,5 % du lien. Décalage minimal : $65\,535 \times 2^4 = 1\,048\,560 \geq 10^6$, donc $s = 4$.

Piège

La p. 16 écrit « window size * segment size (approximately 64k * 1500) ». La p. 3 précise que le champ Window Size compte des **octets** libres, et la fenêtre maximale de 1 Go (2^{30} octets) de la même p. 16 n'est cohérente qu'en octets. Calculer en octets et le dire.

Liens à fort taux de perte (p. 17)

Typique du sans-fil (Wi-Fi, ZigBee). Reno prend toute perte pour de la congestion et réduit sa fenêtre à presque rien. Remèdes : fenêtre pilotée par le délai plutôt que par les pertes ; fast retransmit, très utile ; **SACK**, qui acquitte plusieurs segments d'un coup (New Reno et d'autres diffèrent alors

la réduction de fenêtre, sachant que des acquittements arriveront dans le désordre).

Complément TD 2019, p. 55 : ligne DSL de 10 Mbit/s avec un taux d'erreur binaire de 10^{-6} ; un paquet de 1500 octets fait 12 000 bits, donc environ un paquet sur 83,3 est perdu ($12\,000 \times 10^{-6} = 1{,}2$ %), à comparer à un code correcteur qui coûte 20 % fixes (8 Mbit/s utiles). Pour TCP, ces pertes ressemblent à de la congestion.

44. Le Web : navigateurs, serveurs, HTML, JavaScript

Source : `hta6-applications.pdf`, p. 2 à 7

Couche application (p. 2) : la présentation est à la charge de l'application ; la session est surtout faite par TCP, sauf le keepalive.

- **WWW** (p. 3) : système de partage de documents et médias, aujourd'hui plateforme logicielle dont le document est l'interface. Hypertexte : documents reliés par des liens, identifiés par leur URL, pouvant pointer vers d'autres serveurs, sans détection automatique des liens morts. Projet Xanadu (années 1960), premier projet hypertexte. Pour s'y retrouver : annuaires (Yahoo !), moteurs de recherche avec robots d'indexation ; `robots.txt` désactive l'indexation. Normalisation : W3C.
- **Navigateurs** (p. 4) : « user agent » dans les normes. Moteur = client HTTP, rendu HTML, exécution JavaScript. Presque tous utilisent Blink (Google) : Chrome, Edge, Opera, Brave ; CEF pour des interfaces d'applications. Encore développés : Gecko (Firefox), WebKit (Safari ; Blink en est un fork). Abandonnés : Trident (Internet Explorer), Presto (Opera). Google dicte l'évolution du Web : problème d'interopérabilité et de libre accès.
- **Serveurs** (p. 5) : IIS, Apache, nginx, Lighttpd. Fichiers ordinaires ou programme externe qui génère le contenu ; listage de répertoires (HTTP a remplacé FTP). **Proxy** : intermédiaire entre client et serveur, pour répartition de charge, cache, filtrage, traduction de protocole (Squid, nginx).
- **HTML** (p. 6) : format standard du Web, d'abord mal défini ; XHTML (HTML valide XML), remplacé par HTML5 ; balisage de structure et de forme, mise en forme séparée par CSS.
- **JavaScript** (p. 7) : code exécuté dans le navigateur qui modifie le HTML ; Brendan Eich, Netscape, 1995 ; nom marketing sans lien avec Java, nom normalisé ECMAScript ; DOM, API d'accès au document ; XMLHttpRequest (IE5, 1999) et AJAX ; HTML devient une interface pour tout logiciel (WebGL).

45. HTTP et ses versions

Source : `hta6-applications.pdf`, p. 8 à 10

HyperText Transfer Protocol, protocole **requête et réponse**, **sans état** : chaque requête est indépendante. Méthodes : **GET** récupère l'objet désigné par l'URL ; **POST** la requête contient des données à placer à l'URL (commentaire, recherche, panier) ; **HEAD** récupère les métadonnées. GET et PUT sont si génériques qu'on peut tout faire passer par HTTP (« everything-over-http »).

Classe	Sens, exemples de la slide
1XX	en cours (rare)
2XX	succès : 200 OK
3XX	redirection : 301 Moved Permanently
4XX	erreur du client : 403 Forbidden, 404 Not Found
5XX	erreur du serveur : 500 Internal Server Error, 502 Bad Gateway, 504 Gateway Timeout

La réponse contient un code, un en-tête de métadonnées et le document. 418 I'm a Teapot est un poisson d'avril (RFC 2324).

En-têtes cités (p. 9) : `User-Agent` (le navigateur s'identifie ; à force de contenus adaptés aux bogues de chaque navigateur, les navigateurs mentent sur leur identité, d'où un désordre ingérable) ; `Server` (le serveur s'identifie) ; `Referer` (URL de la page qui contenait le lien : comment les gens trouvent un site).

Version	Norme	Nouveauté
HTTP/0.9	brouillon, 1989	spécification simple
HTTP/1.0	RFC 1945, 1996	première spécification
HTTP/1.1	RFC 2068 (1997), RFC 2616 (1999)	**réutilisation de la connexion TCP** pour plusieurs objets du même serveur
HTTP/2.0	RFC 7540, 2015	issu de SPDY (Google) : compression des en-têtes, **multiplexage des requêtes**
HTTP/3.0	RFC 9114, 2022	**QUIC au lieu de TCP** : contrôle de congestion maison sur UDP, meilleur multiplexage, sécurité intégrée

Ce que change la réutilisation de connexion (rappel général, modèle de Kurose et Ross)

On néglige le temps d'émission. HTTP/1.0 sans réutilisation : chaque objet coûte 1 RTT d'ouverture TCP et 1 RTT de requête et réponse. Page de 5 objets, RTT 50 ms, objets demandés l'un après l'autre : $5 \times 2 = 10$ RTT $= 500$ ms. HTTP/1.1, une seule ouverture : $1 + 5 = 6$ RTT $= 300$ ms.

Rappel général, forme d'une requête HTTP/1.1 : ligne `GET /index.html HTTP/1.1`, puis des en-têtes `Champ: valeur` dont `Host:` (obligatoire en 1.1), une ligne vide, et le corps éventuel. La réponse commence par `HTTP/1.1 200 OK`.

46. État HTTP : cookies

Source : `hta6-applications.pdf`, p. 11

HTTP est sans état : comment une boutique garde-t-elle un panier, sachant que stocker les informations clients sur le serveur coûte cher ?

À retenir

- **Cookies** (Netscape, 1995) : stockage clé et valeur dans le user agent. Une réponse HTTP pose des cookies ; le navigateur les renvoie avec les requêtes suivantes au **même site**. Pour le client, le contenu n'est qu'une suite d'octets qu'il n'interprète pas.
- Usages : session (connexion, authentification), personnalisation (thème), pistage.
- Sécurité : ne renvoyer le cookie qu'au serveur qui l'a posé. **Cookie tiers** : posé par le serveur pour le compte d'un autre domaine.
- Vie privée : pistage par cookies tiers par les serveurs de contenus communs (framework JS, publicité) ; empreinte du navigateur (fingerprinting) même sans cookie.
- Web storage (HTML5) : stockage permanent bien plus grand.
- Cache : une réponse avec cookie ne peut pas être mise en cache, d'où le contenu statique servi depuis un domaine séparé, sans cookie.

47. HTTPS, clés publiques, TLS

Source : `hta6-applications.pdf`, p. 12 à 14

HTTPS : transmission sûre et authentification du serveur. Ports par défaut : HTTP **80**, HTTPS **443**.

Cryptographie	Points de la slide
Symétrique	un secret commun, même clé pour chiffrer et déchiffrer. Diffie-Hellman (1976), exponentielles en arithmétique modulaire : la clé ne passe jamais sur le fil. **Aucune protection contre l'homme du milieu**
Asymétrique	paire clé publique, clé privée. Chiffrer avec la clé publique du destinataire : lui seul déchiffre, protège contre l'homme du milieu. **Signature** : chiffrer avec sa clé privée, tous vérifient avec la clé publique ; lent, donc on signe un hachage sûr (SHA-256). RSA, premier algorithme fonctionnel, toujours utilisé

Publier les clés publiques (p. 13)

Insoluble dans le cas général ; deux approches pratiques.

- **Web of trust** (PGP) : je fais confiance à ceux à qui mes amis de confiance font confiance. Difficulté : trouver des amis bien reliés au réseau de confiance.
- **PKI** : des autorités de certification (CA) dont tout le monde connaît la clé publique signent les clés publiques des sites (après vérification du propriétaire). La clé d'un site est légitime si elle a été **émise pour le site auquel on est connecté** et **signée par une CA de confiance**. Les certificats X.509 portent les clés (encodage ASN.1 DER). Certificat auto-signé : utilisable pour chiffrer, pas pour prouver l'identité du site. Critiques : CA d'un pays étranger, voire hostile, approuvée par défaut ; CA ayant vendu de l'écoute ; comment connaître les clés des CA ?

SSL et TLS (p. 14)

> **À retenir**
>
> - Même technologie : après SSL 3.0 vient TLS 1.0. Tout ce qui est an-

térieur à TLS 1.2 est aujourd'hui déprécié.

- Protocole de sécurité de **couche application**, malgré son nom ; utilise PKI et certificats ; surtout pour HTTPS mais indépendant de lui.
- **Après la poignée de main TCP, une poignée de main TLS.**
- **StartTLS** : inciter le client à passer une connexion non sûre en TLS.
- 99 % des sites sont en HTTPS.

48. QUIC

Source : `hta6-applications.pdf`, p. 15 (figure : blog.apnic.net)

À retenir

- Quick UDP Internet Connections, RFC 9000, nouveauté majeure de HTTP/3.0.
- Protocole « fait maison » sur **UDP**, implémenté en couche application.
- Multiplexage conscient des éléments web, meilleur ordonnancement.
- Session et poignée de main TLS combinées : **3 échanges au lieu de 6**. Chiffrement obligatoire.
- Identifiant de session de transport indépendant des ports : **mobilité**.
- Contrôle de congestion non spécifié (New Reno recommandé ?, selon la slide).
- Vraiment utilisé surtout par Google : pris en charge par tous les navigateurs, mal par les serveurs ; le chiffrement obligatoire gêne les proxys cache ; la redirection de TCP vers QUIC n'est pas clairement définie.

La figure compare TCP + TLS 1.3 (SYN, SYN ACK, ACK + ClientHello, ServerHello…, Client Finished) et QUIC (Initial ClientHello ; Initial ServerHello et certificat ; Client Finished).

Rappel général (RFC 8446, RFC 9000), en RTT avant d'envoyer la requête HTTP sur une nouvelle connexion : TCP + TLS 1.3, 1 RTT pour TCP et 1 pour TLS, donc 2 RTT ; QUIC, 1 RTT. Avec un RTT de 50 ms : 100 ms contre 50 ms.

Piège

La p. 16 (« WWW Alternatives » : Gopher, Gemini) est marquée « Not exam material ».

49. E-mail : architecture, format, stockage

Source : `hta6p-applications.pdf`, p. 3 à 6 (figure p. 4 : unione.io)

Le courrier électronique imite le courrier postal : rédiger, envoyer, arrivée dans une boîte, lecture à tout moment. Origine : boîte par utilisateur sur les systèmes Unix ; adresse `utilisateur@ordinateur`. Aujourd'hui, l'adresse n'est plus liée à un compte sur un serveur : on n'y a en général qu'une boîte.

Architecture (p. 4)

Pas définie formellement par une RFC, simple notation usuelle.

Agent	Rôle
MTA	Message (ou Mail) Transfer Agent : relaie les messages pour d'autres agents (SMTP)
MDA	Mail Delivery Agent : relais qui stocke ; c'est là qu'est la boîte de l'utilisateur
MUA	Mail User Agent : le client de messagerie
MSA, MRA	Message Submission Agent, Message Retrieval Agent

> **À retenir**
>
> Chaîne de la figure p. 4 : l'expéditeur écrit dans son **MUA** → **MSA** (soumission) → **MTA** (relais SMTP) → **MDA** (boîte du destinataire) → **MUA** du destinataire, qui relève le message.

Format (p. 5)

- RFC 5322 (avant : RFC 2822, et RFC 822 dans ARPANET).
- D'abord les en-têtes `Champ: valeur` : Date, From, To, CC, BCC, Subject, Reply-to, In-reply-to, Message-id… Nom affiché : `"Display Name" <email@address>` ; plusieurs destinataires séparés par des virgules.
- Corps après une ligne vide, en lignes de caractères ASCII **7 bits** uniquement.
- **MIME** (RFC 2045) : convertir toute donnée en ASCII 7 bits (caractères internationaux, binaire), encodages quoted-printable ou base64. En-tête

`Content-Type: application/octet-stream` ; type MIME normalisé par l'IANA, par exemple `text/plain`.

- Pièces jointes : corps multipart, toujours encodé en MIME.

Rappel général : base64 code 3 octets en 4 caractères, donc une pièce jointe grossit d'un tiers (un fichier de 3 Mo devient 4 Mo de texte, plus les fins de ligne).

Stockage (p. 6)

mbox : tous les messages concaténés dans un fichier, chaque en-tête commence par `From ` ; format de mailx sous Unix ; RFC 4155 (2005), type `application/mbox`. **maildir** : un fichier par message, nom unique avec hachage ; non normalisé ; sous-dossiers (tri par catégorie) ; dans chaque dossier, trois répertoires `new`, `cur`, `tmp`.

50. SMTP, POP3, IMAP, clients et serveurs

Source : `hta6p-applications.pdf`, p. 7 à 12

Protocole	Norme, ports	Rôle, commandes
SMTP	RFC 821 (1982), RFC 5321 (2008) ; TCP 25 entre serveurs ; soumission 465 (avant 587) ; TLS	entre MTA, et soumission du MUA au MTA ; texte, avec connexion ; `MAIL <from>`, `RCPT <to>`, `DATA` ; codes de réponse organisés comme HTTP ; enregistrement **MX** du DNS pour trouver le MTA d'un domaine
POP3	RFC 1081, puis RFC 1939 ; 110, 995 avec TLS	relève simple, suppression optionnelle sur le serveur : `RETR`, `DELE` ; réponse OK ; boîte de réception seule, pas de sous-dossiers
IMAP	version 4, RFC 9051 ; 143, 993 avec TLS	requêtes sur la base du serveur : plusieurs boîtes en dossiers, en-têtes et pièces jointes séparément, dépôt dans « Sent » après envoi SMTP ; relever ou seulement consulter : `FETCH` ; réponse OK

À retenir

- Le MUA se connecte au **MDA** pour relever (POP3, IMAP) et au **MTA** pour envoyer (SMTP). Exemples : Thunderbird, KMail, Evolution, The Bat ! ; en mode texte mailx, pine, mutt.
- Le **webmail** est en général un client IMAP (SquirrelMail, RoundCube, Citadel).
- MTA cités : Sendmail (classique sous Unix, peut aussi soumettre comme client), Exim (défaut sous Debian), Postfix, Dovecot.
- Microsoft (p. 12) : Exchange Server, Outlook ; protocole propriétaire au lieu de POP3 et IMAP (qu'il prend aussi en charge) ; premier à ajouter agenda et tâches, également propriétaires.

Rappel général (RFC 5321), dialogue SMTP type, client puis réponse du serveur : `EHLO` (250), `MAIL FROM:<a@x>` (250), `RCPT TO:<b@y>` (250), `DATA` (354), le message terminé par une ligne contenant un seul point (250), `QUIT` (221). Dovecot, cité par la slide parmi les MTA, est d'abord un serveur IMAP et POP3, donc côté MDA.

51. Sécurité de l'e-mail : authentification, PGP, spam, phishing

Source : `hta6p-applications.pdf`, p. 13 à 19

Authentification du client (p. 13 et 14)

Se connecter de façon sûre au MTA et au MDA : jamais de mot de passe en clair. Facteurs : ce que l'on sait (mot de passe), ce que l'on a (clé), ce que l'on est (biométrie).

- **PLAIN** : connexion TLS, puis mot de passe en clair dans le tunnel.
- **CRAM** (Challenge-Response Authentication Method) : serveur et client connaissent le mot de passe ; le serveur envoie un aléa, le client renvoie hash(mot de passe, aléa), le serveur calcule le même et compare. Il faut un générateur d'aléa et un hachage sûrs. Protège contre le **rejeu** : chaque connexion a un aléa différent.
- **OpenID** : une seule connexion chez un fournisseur d'identité, que les autres sites interrogent (se connecter avec son compte Google, Microsoft, Steam).
- **OAuth 2.0** (RFC 6749) : le fournisseur donne l'identité et un jeton d'autorisation pour agir au nom de l'utilisateur (Gmail en IMAP et SMTP avec l'authentification Google).
- **TOTP** (RFC 6238) : secret partagé entre le service et l'appareil d'authentification ; nombre à 6 chiffres calculé avec le secret et l'heure, valide moins d'une minute ; sert au 2FA (prouver qu'on a l'appareil) ; exige une horloge assez juste.

PGP (p. 15)

Signer et chiffrer du texte, donc du courrier (Debian APT signe aussi ses paquets avec PGP). Cryptographie asymétrique ; **web of trust** au lieu des certificats X.509 et des CA (on fait confiance à ceux à qui nos personnes de confiance font confiance, relation transitive). Phil Zimmermann, 1991, code ouvert ; exportation interdite depuis les États-Unis dans les années 1990, enquête abandonnée. Norme OpenPGP RFC 4880 ; implémentation GnuPG. **S/MIME** (RFC 8551) pour les pièces jointes signées ou chiffrées, peu prati-

cable en webmail.

Spam (p. 16 à 18)

Spam : courriels non sollicités, publicité ou escroquerie, parfois avec un logiciel malveillant en pièce jointe (le nom vient de la viande en conserve et d'un sketch des Monty Python de 1970). Lutte « généralement impossible » ; tentatives :

Mesure	Principe
Côté MTA	n'accepter que des MUA authentifiés, ou des MTA « respectés » (comment le devenir ?)
DKIM, RFC 6376	le MTA émetteur **signe** le message pour montrer qu'il en juge le contenu légitime ; clés publiques dans le DNS
SPF, RFC 7208	le MTA prouve qu'il est **autorisé à envoyer** pour le domaine de l'en-tête From
DMARC, RFC 7489	publier dans le DNS comment son MTA applique DKIM et SPF
Côté MUA	filtrer : mots-clés, motifs suspects dans les en-têtes ; il faut une forme d'IA pour reconnaître les motifs

Phishing (p. 19)

Ingénierie sociale : tromper les gens pour qu'ils révèlent des informations sensibles ou installent un logiciel malveillant ou un rançongiciel. Très courant : acteurs individuels, étatiques, espionnage industriel. Impossible de s'en protéger totalement : attaques très élaborées, utilisateurs qui cliquent sur un lien par erreur.

52. Méthode d'examen

Source : types d'exercices : `hta2p-ipv4-practical.pdf`, p. 11 à 14 ; `hta1p-protocols.pdf`, p. 26

Format annoncé : 6 questions ouvertes, 12/30 pour réussir. Modèles donnés par les slides : les quatre « Practical Examples » de `hta2p`, la conversion XML vers JSON ou YAML, les déroulés pas à pas de `hta4p` (DV, count to infinity, Dijkstra) et de `hta5p` (fenêtres TCP). Réponse ouverte : écrire la règle, puis l'appliquer, puis conclure ; une justification juste rapporte des points même si le calcul déraille.

À retenir

1. Validité : octets ≤ 255, masque continu, réseau aligné.
2. Convertir le masque en /n (ou l'inverse).
3. Repérer l'octet intéressant et le nombre magique $256 -$ octet.
4. Écrire en binaire l'octet intéressant, aligné avec le masque.
5. Contrôler : réseau multiple de m, broadcast = réseau $+m - 1$, hôtes $= 2^{32-n} - 2$.

Question	Méthode
masque et préfixe	concept 10, compter les 1
réseau, broadcast, hôtes	ET bit à bit ou nombre magique (11, 12)
même sous-réseau ?	comparer les n premiers bits (13)
découper, VLSM	plus grands besoins d'abord, alignement (14, 15)
fusionner des routes	seul le dernier bit diffère, même saut suivant (16)
interface de sortie	tous les préfixes qui correspondent, puis le plus long (17)
décoder un en-tête	0x45, unités des champs (19)
XML vers JSON, YAML	règles du concept 5
Dijkstra	tableau étape par étape, puis premier saut (30)
vecteur de distance	min sur les voisins de coût lien + distance annoncée (31)
count to infinity	dérouler les annonces C et D, split horizon (31)
fenêtre TCP par ronde	slow start ×2, CA +1, timeout ou 3 ACK dupliqués (41, 42)
fenêtre optimale	BDP = débit × RTT, en octets (43)
NAT	tableau source et destination par tronçon (38)
en-tête TCP, UDP	champs de 16 et 32 bits, octet des drapeaux (35, 37)
chaîne d'un e-mail	MUA, MSA, MTA, MDA, protocole et port à chaque étape (49, 50)

Piège

- Oublier le « −2 » des hôtes, confondre dernier hôte et broadcast.
- Fusionner des routes non alignées ou vers des interfaces différentes.
- S'arrêter à la première route qui correspond au lieu de la plus longue.
- Dijkstra : modifier un nœud déjà retiré ; oublier de donner le premier saut.
- TCP : confondre timeout (retour à 1) et 3 ACK dupliqués (Reno :

moitié, on reste en CA).

- BDP : mélanger bits et octets.

53. Formulaire

À retenir

Poids d'un octet	128, 64, 32, 16, 8, 4, 2, 1
Octets de masque	128, 192, 224, 240, 248, 252, 254, 255
Adresses d'un /n	2^{32-n}
Hôtes d'un /n	$2^{32-n} - 2$
Nombre magique	256 moins l'octet du masque
Réseau	adresse ET masque
Broadcast	partie hôte à 1
s bits empruntés	2^s sous-réseaux, préfixe $n + s$
Sous-réseau k	base $+k \times 2^{32-(n+s)}$
Fusion de deux /n	même préfixe sauf le dernier bit, même saut suivant : un $/(n-1)$
Routage	plus long préfixe ; 0.0.0.0/0 en dernier recours

Adresses	
Classes A, B, C	1 à 126 (/8), 128 à 191 (/16), 192 à 223 (/24)
D, E	multicast `1110`, réservé `11110`
Privées	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
Loopback	127.0.0.1 (réseau 127)
En-tête IPv4	
Version, IHL	4 ; IHL × 4 octets, minimum 20
Total Length	20 à 65 535 octets
Fragment Offset	unités de 8 octets ; flags DF, MF
Protocol	ICMP 1, IGMP 2, TCP 6, UDP 17
MTU	Ethernet 1500 ; jumbo 9000
Protocoles	
ICMP	0 Echo Reply, 3 Unreachable, 5 Redirect, 8 Echo Request, 11 Time Exceeded
DHCP	UDP 67 (serveur), 68 (client) ; DORA ; bail
DNS	UDP/53, TCP/53 ; A, AAAA, MX
PCM	8 kHz × 8 bits = 64 kbit/s
Token, leaky bucket	K = moyen ; B = (max −moy)× rafale

Routage	
DV	Bellman-Ford : $d_X(Y) = \min_v(c(X,v) + d_v(Y))$; RIP, IGRP
LS	Dijkstra ; OSPF, IS-IS
RIP	sauts, limite 15 (16 = infini), mise à jour 30 s
OSPF	coût ; aires, dorsale 0.0.0.0 ; DR et BDR
BGP	path vector, politique, entre AS ; iBGP, route reflector
Contre le count to infinity	split horizon, poisoned reverse
Transport	
Protocoles IP	TCP 6, UDP 17, DCCP 33, SCTP 132
Ports	16 bits ; bien connus 0 à 1023 ; 5-uplet
En-têtes	UDP 8 octets ; TCP 20 octets sans option (Data Offset 5)
Poignée de main	SYN (Sc) ; SYN,ACK (Ss, Sc+1) ; ACK (Sc+1, Ss+1)
Fenêtre effective	min(`adv_wnd`, `cwnd`)
Slow start	`cwnd` × 2 par ronde jusqu'à `ssthresh`
Congestion avoidance	`cwnd` + 1 par ronde
Timeout	`ssthresh` /2, `cwnd` = 1, slow start
3 ACK dupliqués (Reno)	fast retransmit, `cwnd` /2, reste en CA
BDP	débit × RTT ; débit max = W/RTT
Fenêtre max	65 535 octets ; $\times 2^{14}$ avec window scale (1 Go)
Applications	
HTTP, HTTPS	80, 443 ; 1.1 réutilise TCP, 2 multiplexe, 3 sur QUIC (UDP)
QUIC	RFC 9000 ; TLS intégré, 3 échanges au lieu de 6
SMTP	25 entre serveurs ; soumission 465 (avant 587)
POP3, IMAP	110 et 995 ; 143 et 993
Chaîne d'e-mail	MUA, MSA, MTA, MDA, MUA
Anti-spam	DKIM signe, SPF autorise, DMARC publie la politique